Laporan Tugas 2 IF4020 Kriptografi

Perancangan Block Cipher Baru

Pregnant_Sister

Block Cipher 256-bit Berbasis Cellular Automaton (Conway's Game of Life) dan Jaringan Feistel Hibrida



Disusun oleh:

M Rayhan Farrukh	13523035
Darrel Adinarya S	13523061
Rhio Bimo Prakoso S	13523123
Frederiko Eldad M	13523147

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



M Rayhan Farrukh (13523035)



Darrel Adinarya Sunanda (13523061)



Rhio Bimo Prakoso S (13523123)



Frederiko Eldad Mugiyono (13523147)

Daftar Isi

1 Pendahuluan **1**
- 1.1 Latar Belakang . . . 1
- 1.2 Tinjauan Singkat Block Cipher Pembanding . . . 1
  - 1.2.1 Data Encryption Standard (DES) . . . 1
  - 1.2.2 Advanced Encryption Standard (AES) . . . 2
  - 1.2.3 RC6 . . . 2
  - 1.2.4 Perbandingan . . . 2
- 1.3 Gagasan Utama Rancangan . . . 2
- 1.4 Tujuan . . . 3
- 1.5 Sistematika Laporan . . . 3

2 Dasar Teori **4**
- 2.1 Block Cipher, Confusion, dan Diffusion . . . 4
- 2.2 Jaringan Feistel . . . 4
- 2.3 Substitution–Permutation Network dan Komponen AES . . . 4
- 2.4 Cellular Automaton dan Conway's Game of Life . . . 5
- 2.5 Related Works: Cellular Automaton dalam Kriptografi . . . 5
- 2.6 Mode Operasi dan Padding . . . 5
- 2.7 Metrik Pengujian Keamanan . . . 5

3 Perancangan Block Cipher **7**
- 3.1 Spesifikasi Umum . . . 7
- 3.2 Alur Global Enkripsi dan Dekripsi . . . 8
- 3.3 Jaringan Feistel 18 Ronde . . . 8
- 3.4 Fungsi Ronde f_{GoL} . . . 9
  - 3.4.1 Implementasi Game of Life Secara Bit-Paralel . . . 10
- 3.5 Fungsi Ronde f_{SPN} . . . 11
  - 3.5.1 Susunan State . . . 11
  - 3.5.2 Kotak-S (SubBytes) . . . 11
  - 3.5.3 ShiftRows dan MixColumns . . . 12
  - 3.5.4 Penjumlahan Modular dan Rotasi Bergantung Data . . . 12
- 3.6 Penjadwalan Kunci (Round Key) . . . 13
- 3.7 Dekripsi . . . 15
- 3.8 Mode Operasi . . . 16
- 3.9 Padding dan Format Ciphertext . . . 16

4 Implementasi dan Simulasi **17**
- 4.1 Lingkungan dan Struktur Program . . . 17
- 4.2 Antarmuka Program . . . 17
- 4.3 Skenario Pengujian . . . 19

4.4 Hasil Enkripsi dan Dekripsi . . . 19
4.4.1 Berkas Teks . . . 19
4.4.2 Berkas Biner . . . 19
4.4.3 Enkripsi Probabilistik . . . 20
4.5 Analisis Penanganan Padding . . . 20
4.6 Verifikasi Integritas Data . . . 21
4.6.1 Kecocokan Hash . . . 21
4.6.2 Deteksi Kunci atau Mode yang Salah . . . 21
4.6.3 Deteksi Perubahan Ciphertext . . . 21

5 Analisis Keamanan 23
5.1 Avalanche Effect . . . 23
5.1.1 Tingkat Blok . . . 23
5.1.2 Difusi terhadap Jumlah Ronde . . . 23
5.1.3 Tingkat Berkas pada Setiap Mode . . . 24
5.2 Entropi . . . 25
5.3 Histogram . . . 25
5.4 Pengujian Tambahan . . . 27
5.4.1 Kebocoran Pola pada Mode ECB . . . 27
5.4.2 Analisis Kotak-S dan Iterasi Desain . . . 28
5.4.3 Batas Kriptanalisis Diferensial . . . 28
5.4.4 Iterasi Desain Penjadwalan Kunci: Distribusi Bit Round Key . . . 29
5.4.5 Iterasi Desain Penjadwalan Kunci: Kunci Ekuivalen . . . 30
5.4.6 Integritas dan Malleability . . . 31
5.4.7 Kinerja . . . 31
5.5 Ringkasan Hasil Analisis . . . 32

6 Kesimpulan dan Saran Pengembangan 33
6.1 Kesimpulan . . . 33
6.2 Saran Pengembangan (Future Works) . . . 33

Daftar Pustaka 35

A Tautan Repositori dan Dokumentasi 37

B Keluaran Pengujian Keamanan 38

C Hash SHA-256 Berkas Uji 39

D Kode Sumber Inti 40

Daftar Gambar

3.1 Alur global enkripsi Pregnant_Sister 8
3.2 Struktur jaringan Feistel 18 ronde pada Pregnant_Sister 9
3.3 Dua jenis fungsi ronde pada Pregnant_Sister 10
3.4 Pohon penjumlah untuk menghitung jumlah tetangga 16 sel sekaligus (FA = *full adder*, HA = *half adder*; garis putus-putus = *carry*) 11
3.5 Alur penjadwalan kunci berbasis Game of Life 13
3.6 Urutan pengisian spiral grid 16×16. Garis oranye menunjukkan urutan sel, dan warna menunjukkan putaran materi kunci ke-ℓ (32 sel per putaran). 14
3.7 Contoh tahapan penjadwalan kunci pada ronde pertama. Sel hitam bernilai 1 (hidup); panel (a) menampilkan nilai byte 0–255 dalam skala abu-abu. Panel (f) adalah grid yang dilipat menjadi K_1. 15
3.8 Format *ciphertext* Pregnant_Sister 16

4.1 Halaman dokumentasi API Pregnant_Sister 18

5.1 Persentase bit *ciphertext* yang berubah terhadap jumlah ronde yang dijalankan (12 blok $\times$ 64 posisi bit per titik) 24
5.2 Histogram `glasses.txt`: *plaintext* (kiri) dan *ciphertext* CBC (kanan). Garis putus-putus menunjukkan frekuensi harapan distribusi seragam. 26
5.3 Histogram `image.jpg`: *plaintext* (kiri) dan *ciphertext* CBC (kanan) 26
5.4 Histogram `video.mp4`: *plaintext* (kiri) dan *ciphertext* CBC (kanan) 26
5.5 Citra mentah yang dienkripsi dengan ECB, CBC, dan CTR (setiap baris citra 256 piksel = 8 blok) 28
5.6 Proporsi bit 1 pada grid dan *round key* untuk setiap ronde penjadwalan kunci (300 pasangan kunci dan *nonce* acak; angka di kanan = rata-rata 18 ronde; garis putus-putus = 0,5) 30

Daftar Tabel

1.1 Perbandingan Pregnant_Sister dengan *block cipher* sejenis 2

3.1 Spesifikasi Pregnant_Sister 7
3.2 Notasi 7
3.3 Kotak-S 8×8 bit yang dipakai pada f_{SPN} (nilai heksadesimal; baris = 4 bit atas, kolom = 4 bit bawah masukan) 12
3.4 Susunan *state* 4×4 sebelum dan sesudah `ShiftRows` (indeks byte masukan) . . 12
3.5 Urutan ronde pada enkripsi dan dekripsi 15
3.6 Mode operasi yang didukung (E/D = enkripsi/dekripsi satu blok, $C_0 = O_0 = N$) 16

4.1 Struktur modul pada paket `src` 17
4.2 Berkas uji 19
4.3 Hasil enkripsi–dekripsi `glasses.txt` (SHA-256 *plaintext* = `76d3bcab876abc41...`) 19
4.4 Hasil enkripsi–dekripsi `image.jpg` (SHA-256 *plaintext* = `5a92b5ae1f92711d...`) 20
4.5 Hasil enkripsi–dekripsi `video.mp4` (SHA-256 *plaintext* = `f8564e7de424642c...`) 20
4.6 Perilaku *padding* PKCS#7 (blok 32 byte) pada berbagai panjang *plaintext* 20
4.7 Deteksi dekripsi dengan kunci atau mode yang salah (400 percobaan per skenario) 21
4.8 Propagasi galat akibat pembalikan 1 bit pada blok *ciphertext* C_2 (jumlah bit berubah dari 256 bit per blok *plaintext*) 22

5.1 *Avalanche effect* tingkat blok (256 bit) 23
5.2 *Avalanche effect* tingkat berkas per mode operasi 24
5.3 Entropi Shannon (bit per byte, maksimum 8) *plaintext* dan *ciphertext* 25
5.4 Statistik χ^2 histogram (lolos jika $< 293{,}25$) 27
5.5 Distribusi χ^2 dari enkripsi ulang dengan *nonce* berbeda 27
5.6 Perbandingan Kotak-S versi awal dan versi akhir 28
5.7 Hasil pengukuran kinerja 31
5.8 Ringkasan hasil analisis keamanan 32

C.1 *Hash* SHA-256 berkas uji 39

Bab 1

Pendahuluan

1.1 Latar Belakang

Block cipher adalah primitif kriptografi kunci simetris yang mengenkripsi data dalam satuan blok berukuran tetap. Hampir semua protokol pengamanan data modern, mulai dari TLS, enkripsi diska, hingga pesan instan, bergantung pada *block cipher* yang dipadukan dengan mode operasi tertentu. Keamanan sebuah *block cipher* ditentukan oleh seberapa baik ia mewujudkan dua prinsip Shannon [19], yaitu *confusion* (hubungan antara kunci dan *ciphertext* dibuat serumit mungkin) dan *diffusion* (pengaruh satu bit *plaintext* disebar ke sebanyak mungkin bit *ciphertext*).

Rancangan *block cipher* yang dipakai saat ini umumnya menggunakan komponen yang sudah dipelajari secara mendalam, seperti Kotak-S, permutasi, dan operasi aritmetika modular. Di sisi lain, *cellular automaton* (CA) telah lama diusulkan sebagai sumber ketidakteraturan untuk kriptografi [23]. CA terdiri atas sel-sel sederhana yang diperbarui berdasarkan aturan lokal, tetapi dapat menghasilkan perilaku global yang kompleks dan sulit diprediksi. Conway's Game of Life (GoL) [6] adalah CA dua dimensi yang paling dikenal. Aturannya sangat sederhana (B3/S23), tetapi perilakunya cukup kompleks sehingga GoL terbukti *Turing-complete*.

Tugas ini meminta perancangan sebuah *block cipher* baru beserta implementasi, simulasi pada berbagai mode operasi, dan analisis keamanannya. Kelompok kami merancang **Pregnant_Sister**, yaitu *block cipher* 256-bit yang memadukan GoL dengan komponen *block cipher* klasik di dalam jaringan Feistel. Laporan ini menjelaskan rancangan tersebut, hasil pengujiannya, serta kelebihan dan kelemahan yang kami temukan.

1.2 Tinjauan Singkat Block Cipher Pembanding

1.2.1 Data Encryption Standard (DES)

DES distandardisasi oleh NIST pada tahun 1977 [14]. DES memproses blok 64 bit dengan kunci efektif 56 bit melalui 16 ronde jaringan Feistel. Fungsi ronde DES terdiri atas ekspansi 32 bit menjadi 48 bit, XOR dengan *round key*, delapan Kotak-S berukuran 6×4 bit, dan permutasi P. Struktur Feistel membuat proses dekripsi cukup dilakukan dengan menjalankan algoritma yang sama dengan urutan *round key* terbalik. Kelemahan utama DES adalah ruang kuncinya yang kecil (2^{56}), sehingga kunci DES sudah dapat ditemukan dengan pencarian menyeluruh menggunakan perangkat keras khusus pada akhir 1990-an. DES juga rentan secara teoretis terhadap kriptanalisis diferensial [2] dan linear [10].

1.2.2 Advanced Encryption Standard (AES)

AES (Rijndael) [15, 3] menggantikan DES sejak 2001. AES memproses blok 128 bit dengan kunci 128, 192, atau 256 bit dalam 10, 12, atau 14 ronde. Berbeda dengan DES, AES menggunakan struktur *Substitution–Permutation Network* (SPN). Setiap ronde terdiri atas `SubBytes` (Kotak-S 8×8 bit berbasis invers di GF(2^8)), `ShiftRows`, `MixColumns` (perkalian matriks MDS di GF(2^8)), dan `AddRoundKey`. Desain AES berlandaskan *wide trail strategy* yang menjamin batas bawah jumlah Kotak-S aktif, sehingga peluang karakteristik diferensial dan linear dapat dibuktikan cukup kecil.

1.2.3 RC6

RC6 [16] adalah finalis kompetisi AES yang memproses blok 128 bit dalam 20 ronde. Ciri khas RC6 (dan pendahulunya, RC5) adalah *data-dependent rotation*: besar rotasi bit ditentukan oleh nilai data itu sendiri. RC6 juga memadukan operasi dari struktur aljabar yang berbeda, yaitu XOR, penjumlahan modulo 2^w, dan perkalian bilangan bulat, sehingga sulit dianalisis dengan satu kerangka aljabar saja.

1.2.4 Perbandingan

Tabel 1.1 merangkum perbandingan ketiga *block cipher* di atas dengan rancangan Pregnant_Sister.

Tabel 1.1: Perbandingan Pregnant_Sister dengan *block cipher* sejenis

Aspek	**DES**	**AES**	**RC6**	**Pregnant_Sister**
Ukuran blok	64 bit	128 bit	128 bit	256 bit
Panjang kunci	56 bit	128/192/256 bit	128/192/256 bit	$\geq$ 256 bit (di-*hash* SHA-256 menjadi 256 bit)
Struktur	Feistel	SPN	Feistel tergeneralisasi	Feistel dengan dua jenis fungsi ronde
Jumlah ronde	16	10/12/14	20	18 (2 GoL + 16 SPN)
Komponen nonlinear	8 Kotak-S 6 × 4	Kotak-S 8 × 8	Rotasi bergantung data, perkalian	GoL, Kotak-S 8 × 8, penjumlahan mod 256, rotasi bergantung data
Penjadwalan kunci	Permutasi dan rotasi	Ekspansi dengan Kotak-S	Ekspansi dengan konstanta	SHA-256 dan evolusi GoL dengan injeksi ulang *nonce* dan *whitening*
Round key per pesan	Tetap	Tetap	Tetap	Berbeda (bergantung *nonce*)

1.3 Gagasan Utama Rancangan

Rancangan Pregnant_Sister berangkat dari pertanyaan: *bagaimana memanfaatkan sifat kompleks Game of Life tanpa bergantung sepenuhnya pada GoL sebagai satu-satunya sumber kea-*

manan? Evolusi GoL tidak dapat dibalik (banyak konfigurasi berbeda dapat berevolusi ke konfigurasi yang sama). Sifat ini mustahil diterapkan pada SPN murni, tetapi justru tidak menjadi masalah di dalam jaringan Feistel, karena fungsi ronde Feistel tidak perlu memiliki invers. Berdasarkan pengamatan tersebut, kami menyusun lima gagasan utama.

1. **Jaringan Feistel hibrida.** Blok 256 bit dibagi menjadi dua paruh 128 bit dan diproses dalam 18 ronde Feistel. Ronde pertama dan terakhir memakai fungsi ronde berbasis GoL (f_{GoL}), sedangkan ronde 2 sampai 17 memakai fungsi ronde SPN bergaya AES (f_{SPN}).
2. **GoL di dalam fungsi ronde.** f_{GoL} menyusun paruh blok sebagai grid 8×16 bit, lalu mengevolusikannya beberapa generasi pada grid toroidal. Untuk mengurangi tabrakan akibat sifat GoL yang tidak injektif, hasil evolusi di-XOR kembali dengan masukan.
3. **Inti SPN yang teruji.** f_{SPN} memakai Kotak-S, `ShiftRows`, dan `MixColumns` AES yang sifat diferensial dan linearnya sudah terbukti baik, lalu ditambah penjumlahan modulo 256 dengan kunci dan rotasi 128 bit yang besarnya bergantung pada data (terinspirasi RC6).
4. **Penjadwalan kunci berbasis GoL.** Kunci di-*hash* dengan SHA-256, dicampur dengan *nonce*, diisikan secara spiral ke grid 16×16, lalu dievolusikan dengan GoL. Pada setiap ronde, pola baru dari kunci dan *nonce* yang digeser diinjeksikan ulang ke grid sebelum evolusi, dan pola yang sama dipakai kembali untuk *whitening* hasil evolusi sebelum dilipat menjadi *round key*. Dengan demikian, 18 *round key* saling berbeda dan bit-bitnya seimbang.
5. **Round key unik per pesan.** *Nonce* acak 32 byte dibangkitkan setiap kali enkripsi dan ikut menentukan *round key*. Akibatnya, *plaintext* dan kunci yang sama menghasilkan *ciphertext* yang berbeda pada setiap enkripsi, bahkan pada mode ECB.

1.4 Tujuan

Tujuan pengerjaan tugas ini adalah:

1. merancang *block cipher* baru yang memenuhi prinsip *confusion* dan *diffusion*;
2. mengimplementasikan algoritma tersebut beserta lima mode operasi (ECB, CBC, CFB, OFB, CTR) dan mekanisme *padding*;
3. menguji kebenaran enkripsi dan dekripsi pada berkas teks dan biner; serta
4. menganalisis keamanan rancangan melalui uji *avalanche effect*, entropi, histogram, dan pengujian tambahan.

1.5 Sistematika Laporan

Bab 2 membahas dasar teori yang mendasari rancangan. Bab 3 menjelaskan rancangan Pregnant_Sister secara rinci. Bab 4 memaparkan implementasi dan hasil simulasi pada seluruh mode operasi. Bab 5 menganalisis keamanan rancangan. Bab 6 berisi kesimpulan dan saran pengembangan.

Bab 2

Dasar Teori

2.1 Block Cipher, Confusion, dan Diffusion

Block cipher adalah pasangan fungsi $E_K : \{0,1\}^n \to \{0,1\}^n$ dan $D_K = E_K^{-1}$ yang memetakan blok *plaintext* berukuran n bit ke blok *ciphertext* berukuran sama di bawah kunci K [12]. Shannon [19] merumuskan dua prinsip perancangan:

- ***Confusion***: hubungan statistik antara kunci dan *ciphertext* dibuat serumit mungkin, umumnya dengan substitusi nonlinear (Kotak-S).
- ***Diffusion***: pengaruh setiap bit *plaintext* dan kunci disebar ke banyak bit *ciphertext*, umumnya dengan permutasi dan transformasi linear.

Keduanya diwujudkan dengan mengulang fungsi ronde sederhana sebanyak beberapa kali (*product cipher*).

2.2 Jaringan Feistel

Jaringan Feistel [5] membagi blok menjadi paruh kiri L dan kanan R. Pada ronde ke-i dengan *round key* K_i:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus F(R_{i-1}, K_i). \tag{2.1}$$

Dekripsi menggunakan struktur yang sama dengan urutan *round key* terbalik, karena $L_{i-1} = R_i \oplus F(L_i, K_i)$. Sifat terpenting struktur ini adalah **fungsi F tidak perlu memiliki invers**. Fungsi apa pun, termasuk yang tidak injektif, tetap menghasilkan permutasi yang dapat dibalik. DES adalah contoh klasik *cipher* Feistel [14].

2.3 Substitution–Permutation Network dan Komponen AES

SPN menerapkan substitusi dan permutasi secara langsung pada seluruh *state*, sehingga setiap lapisan harus dapat dibalik. AES [15, 3] memakai empat transformasi pada *state* 4×4 byte:

- `SubBytes`: setiap byte x diganti dengan $S(x) = A \cdot x^{-1} \oplus \texttt{0x63}$, dengan x^{-1} invers perkalian di $\mathrm{GF}(2^8)$ modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ dan A matriks afin. Kotak-S ini memiliki keseragaman diferensial 4 (peluang diferensial maksimum 2^{-6}) dan bias linear maksimum 2^{-4}.
- `ShiftRows`: baris ke-i dirotasi ke kiri sejauh i byte.

- `MixColumns`: setiap kolom dikalikan dengan matriks MDS $\begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix}$ di GF(2^8), dengan *branch number* 5.
- `AddRoundKey`: XOR *state* dengan *round key*.

2.4 Cellular Automaton dan Conway's Game of Life

Cellular automaton (CA) adalah sistem dinamis diskret berupa kisi sel yang masing-masing memiliki keadaan berhingga. Semua sel diperbarui serentak menurut aturan lokal yang bergantung pada sel itu sendiri dan tetangganya. Game of Life (GoL) [6] adalah CA dua dimensi biner dengan ketetanggaan Moore (8 tetangga) dan aturan B3/S23:

$$s_{t+1} = \begin{cases} 1 & \text{jika } N_t = 3, \text{ atau } s_t = 1 \text{ dan } N_t = 2, \\ 0 & \text{selainnya,} \end{cases} \tag{2.2}$$

dengan s_t keadaan sel dan N_t jumlah tetangga yang hidup pada generasi t. Pada grid berhingga, batas toroidal (sisi kiri bersambung dengan sisi kanan, sisi atas dengan sisi bawah) membuat setiap sel memiliki tepat delapan tetangga. GoL bersifat **tidak dapat dibalik**: banyak konfigurasi tidak memiliki pendahulu (*Garden of Eden*) dan banyak konfigurasi berbeda berevolusi ke keadaan yang sama.

2.5 Related Works: Cellular Automaton dalam Kriptografi

Wolfram [23] mengusulkan CA satu dimensi aturan 30 sebagai pembangkit bilangan acak semu untuk *stream cipher*, dengan alasan bahwa evolusinya tampak acak walaupun aturannya sederhana. Nandi dkk. [13] kemudian mengusulkan *block cipher* dan *stream cipher* berbasis CA yang dapat diprogram. Secara umum, penelitian kriptografi berbasis CA memanfaatkan dua sifat: paralelisme (semua sel dapat dihitung serentak) dan perilaku kompleks yang muncul dari aturan sederhana. Kelemahan yang sering dilaporkan adalah struktur linear pada aturan tertentu dan kecenderungan konfigurasi menuju keadaan stabil. Kedua kelemahan ini menjadi pertimbangan dalam rancangan kami. GoL dipilih karena aturannya nonlinear, dan GoL tidak dijadikan satu-satunya sumber nonlinearitas.

2.6 Mode Operasi dan Padding

Mode operasi menentukan cara *block cipher* dipakai untuk pesan yang lebih panjang dari satu blok [4]. Pada ECB, setiap blok dienkripsi secara independen. CBC meng-XOR setiap blok *plaintext* dengan blok *ciphertext* sebelumnya. CFB, OFB, dan CTR mengubah *block cipher* menjadi *stream cipher* dengan cara mengenkripsi umpan balik atau *counter*, lalu meng-XOR hasilnya dengan *plaintext*. Mode ECB dan CBC membutuhkan *padding* agar panjang pesan menjadi kelipatan ukuran blok. Skema PKCS#7 [8] menambahkan p byte bernilai p, dengan $1 \le p \le b$ untuk ukuran blok b byte.

2.7 Metrik Pengujian Keamanan

- ***Avalanche effect***: perubahan satu bit masukan (*plaintext* atau kunci) idealnya mengubah sekitar 50% bit keluaran. Kriteria yang lebih ketat, *Strict Avalanche Criterion* (SAC) [22], mensyaratkan setiap bit keluaran berubah dengan peluang tepat $\frac{1}{2}$.
- **Entropi Shannon** [18]: $H = -\sum_{i=0}^{255} p_i \log_2 p_i$ bit per byte, dengan p_i frekuensi relatif nilai byte i. Nilai maksimum 8 dicapai saat seluruh nilai byte muncul dengan peluang sama.

- **Histogram dan uji χ^2**: histogram *ciphertext* yang baik tampak rata. Kerataannya dapat diuji dengan $\chi^2 = \sum_i (O_i - E_i)^2 / E_i$ dengan $E_i = N/256$. Pada 255 derajat kebebasan dan taraf nyata 5%, nilai kritisnya adalah 293,25.
- **Keseragaman diferensial dan linear** Kotak-S [2, 10, 7]: nilai maksimum pada *Difference Distribution Table* (DDT) dan *Linear Approximation Table* (LAT). Semakin kecil nilainya, semakin tahan Kotak-S terhadap kriptanalisis diferensial dan linear.

Bab 3

Perancangan Block Cipher

3.1 Spesifikasi Umum

Pregnant_Sister adalah *block cipher* kunci simetris dengan spesifikasi seperti pada Tabel 3.1. Notasi yang dipakai di seluruh bab ini dirangkum pada Tabel 3.2.

Tabel 3.1: Spesifikasi Pregnant_Sister

Parameter	**Nilai**
Ukuran blok	256 bit (32 byte), dibagi menjadi dua paruh 128 bit
Panjang kunci	minimal 256 bit (32 byte); kunci di-*hash* dengan SHA-256 menjadi 256 bit
Nonce	256 bit (32 byte) acak dari CSPRNG, dibangkitkan setiap enkripsi
Struktur	Jaringan Feistel 18 ronde
Fungsi ronde	f_{GoL} pada ronde 1 dan 18, f_{SPN} pada ronde 2–17
Round key	18 buah, masing-masing 128 bit
Penjadwalan kunci	SHA-256, lalu Game of Life 16×16 toroidal, 4 generasi per ronde, dengan injeksi ulang *nonce* dan *whitening*
Mode operasi	ECB, CBC (bawaan), CFB, OFB, CTR
Padding	PKCS#7 dengan ukuran blok 32 byte
Keluaran	Base64($nonce \,\|\|\, C$)

Tabel 3.2: Notasi

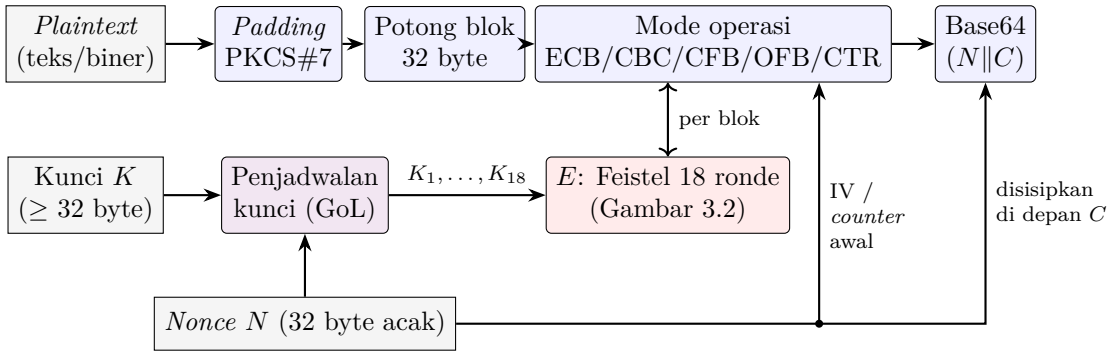
Notasi	**Arti**
$\oplus$	XOR bit demi bit
$\boxplus$	penjumlahan per byte modulo 256
$x \lll n$	rotasi kiri x sejauh n bit
$a \,\|\| \, b$	penyambungan (konkatenasi) a dan b
L_i, R_i	paruh kiri dan kanan (masing-masing 128 bit) setelah ronde ke-i
K_i	*round key* ke-i (128 bit), $1 \le i \le 18$
N	*nonce* 32 byte
$\text{GoL}^g(G)$	hasil evolusi grid G sebanyak g generasi Game of Life

3.2 Alur Global Enkripsi dan Dekripsi

Gambar 3.1 memperlihatkan alur lengkap enkripsi sebuah pesan. Langkah-langkahnya adalah sebagai berikut.

1. Kunci divalidasi (minimal 32 byte) dan data dikonversi menjadi *bytes*.
2. Data diberi *padding* PKCS#7 hingga panjangnya kelipatan 32 byte (Subbab 3.9).
3. *Nonce* 32 byte dibangkitkan secara acak.
4. Penjadwalan kunci menurunkan 18 *round key* dari kunci dan *nonce* (Subbab 3.6).
5. Data dipotong menjadi blok 32 byte, lalu dienkripsi sesuai mode operasi (Subbab 3.8). Setiap pemanggilan E menjalankan 18 ronde Feistel (Subbab 3.3).
6. *Nonce* disambungkan di depan *ciphertext*, lalu hasilnya dikodekan dengan Base64.

Dekripsi menjalankan langkah sebaliknya: Base64 didekode, 32 byte pertama dibaca sebagai *nonce*, *round key* diturunkan ulang, setiap blok didekripsi sesuai mode, lalu *padding* dibuang dan divalidasi.



Gambar 3.1: Alur global enkripsi Pregnant_Sister

3.3 Jaringan Feistel 18 Ronde

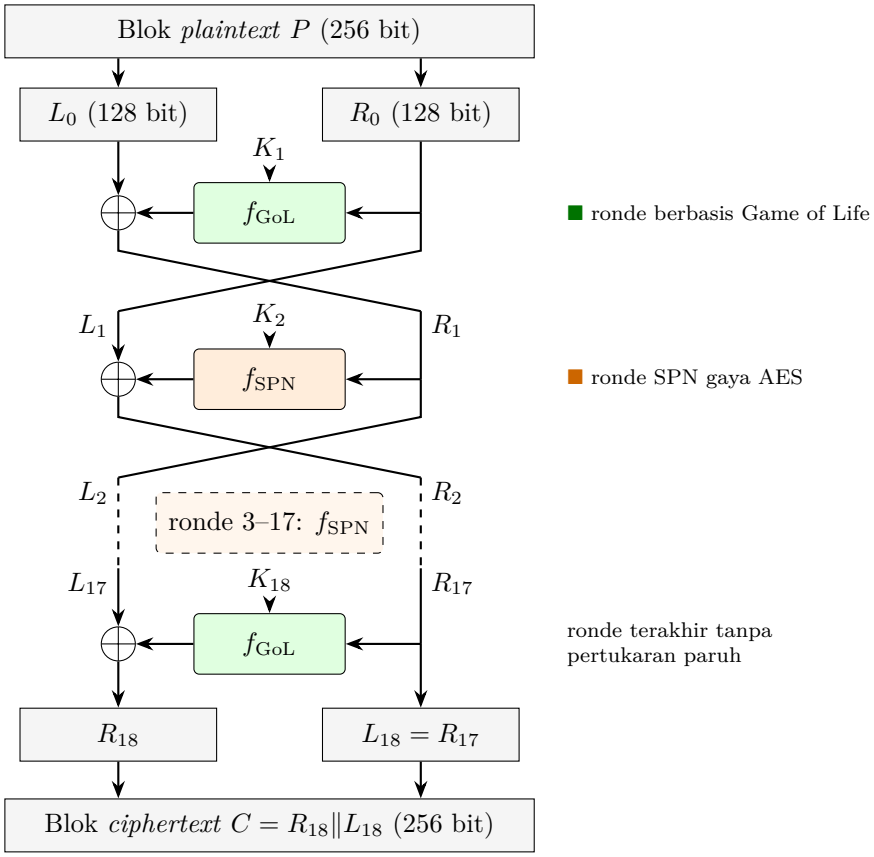
Fungsi enkripsi satu blok E memecah blok 256 bit P menjadi $L_0 \| R_0$ (masing-masing 128 bit), lalu menjalankan 18 ronde Feistel:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus F_i(R_{i-1}, K_i), \qquad F_i = \begin{cases} f_{\text{GoL}} & i \in \{1, 18\}, \\ f_{\text{SPN}} & 2 \le i \le 17. \end{cases} \tag{3.1}$$

Keluaran blok adalah $C = R_{18} \| L_{18}$, yaitu dengan pertukaran paruh terakhir dibatalkan seperti pada DES. Struktur ini diperlihatkan pada Gambar 3.2.

Pemilihan fungsi ronde mengikuti peran masing-masing:

- **Ronde 1 dan 18 (f_{GoL})** berfungsi sebagai lapisan masuk dan keluar yang nonlinear dan berbeda sifat aljabarnya dari inti SPN. Penyerang yang ingin menganalisis inti SPN harus terlebih dahulu melewati transformasi GoL yang tidak memiliki deskripsi aljabar sederhana.
- **Ronde 2–17 (f_{SPN})** memberikan jaminan *confusion* dan *diffusion* yang terukur melalui Kotak-S dan `MixColumns` AES.



Gambar 3.2: Struktur jaringan Feistel 18 ronde pada Pregnant_Sister

3.4 Fungsi Ronde f_{GoL}

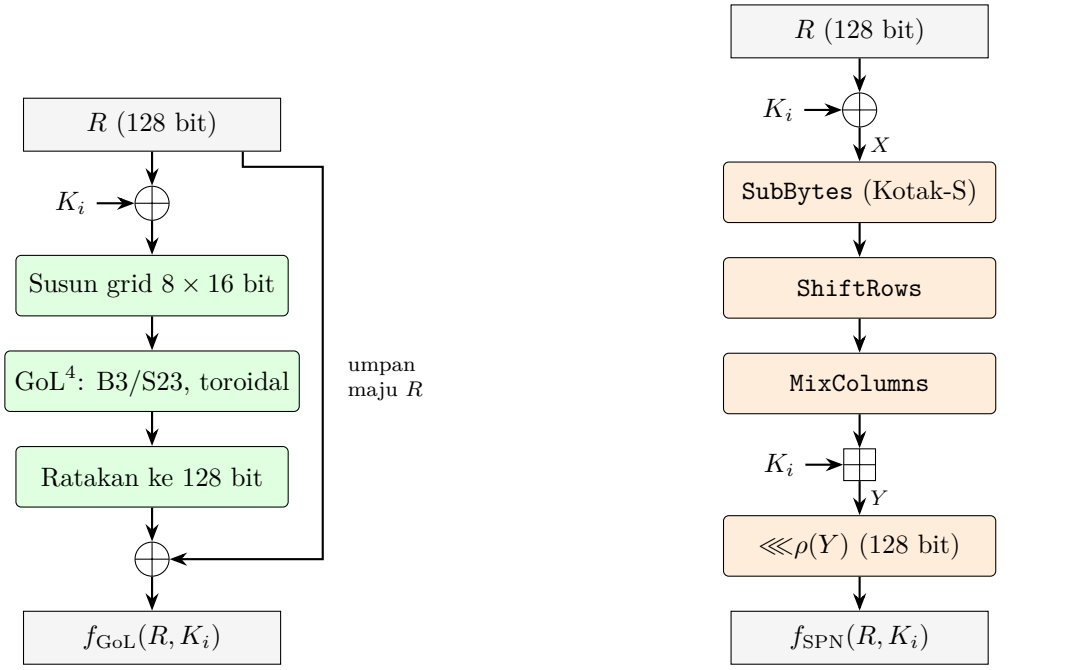
Fungsi ronde berbasis Game of Life didefinisikan sebagai

$$f_{\text{GoL}}(R, K) = \text{Ratakan}\big(\text{GoL}^4(\text{Grid}_{8\times16}(R \oplus K))\big) \oplus R. \tag{3.2}$$

Langkah-langkahnya (Gambar 3.3a) adalah:

1. **Pencampuran kunci.** $X = R \oplus K$ (128 bit).
2. **Penyusunan grid.** 128 bit X disusun menjadi grid 8 baris × 16 kolom secara *big-endian*: dua byte pertama menjadi baris teratas, dan bit paling signifikan menjadi kolom paling kiri.
3. **Evolusi.** Grid dievolusikan 4 generasi dengan aturan B3/S23 (Persamaan (2.2)) pada batas toroidal.
4. **Perataan dan XOR umpan maju.** Grid hasil evolusi diratakan kembali menjadi 128 bit, lalu di-XOR dengan R.

XOR umpan maju pada langkah terakhir penting karena evolusi GoL tidak injektif. Contoh ekstremnya, semua grid yang selnya mati atau terisolasi berevolusi menjadi grid kosong. Tanpa XOR dengan R, banyak nilai R berbeda akan menghasilkan keluaran F yang sama. Dengan XOR tersebut, dua masukan $R \neq R'$ yang tabrakan pada GoL tetap menghasilkan keluaran yang berbeda, karena selisihnya $R \oplus R'$ muncul langsung di keluaran.



(a) Fungsi ronde f_{GoL} (ronde 1 dan 18) (b) Fungsi ronde f_{SPN} (ronde 2–17)

Gambar 3.3: Dua jenis fungsi ronde pada Pregnant_Sister

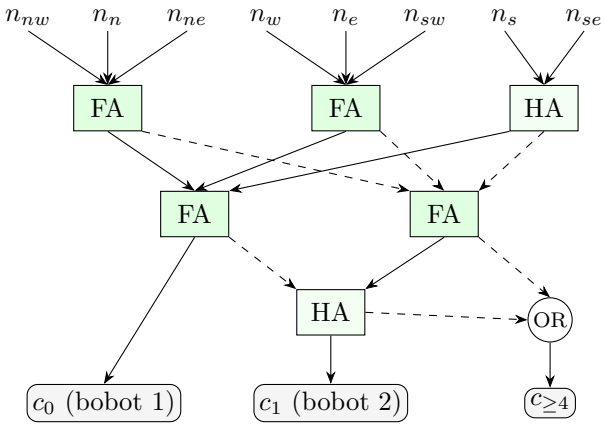
3.4.1 Implementasi Game of Life Secara Bit-Paralel

Setiap baris grid disimpan sebagai satu bilangan bulat (*bitmask*), sehingga satu generasi GoL dihitung per baris dengan operasi bitwise, bukan per sel. Untuk baris r, delapan tetangga diperoleh dari baris di atas, baris itu sendiri, dan baris di bawahnya (indeks modulo jumlah baris), masing-masing dalam versi asli serta versi yang dirotasi satu bit ke kiri dan ke kanan. Rotasi ini mewujudkan batas toroidal pada arah horizontal.

Kedelapan *bitmask* tetangga dijumlahkan secara paralel untuk 16 sel sekaligus menggunakan pohon penjumlah ala Wallace [21] (Gambar 3.4). Hasilnya berupa tiga *bitmask*: c_0 (bit ke-0 jumlah tetangga), c_1 (bit ke-1), dan $c_{\geq 4}$ (jumlah tetangga ≥ 4). Aturan B3/S23 kemudian cukup dihitung dengan

$$s' = \underbrace{(c_1 \wedge \neg c_{\geq 4})}_{N \in \{2,3\}} \wedge (c_0 \vee s), \tag{3.3}$$

yang secara aljabar sama dengan $(N = 3) \vee (s \wedge N \in \{2, 3\})$.



Gambar 3.4: Pohon penjumlah untuk menghitung jumlah tetangga 16 sel sekaligus (FA = *full adder*, HA = *half adder*; garis putus-putus = *carry*)

3.5 Fungsi Ronde f_{SPN}

Fungsi ronde SPN didefinisikan sebagai

$$\begin{aligned} X &= R \oplus K, \\ Y &= \text{MixColumns}\big(\text{ShiftRows}(\text{SubBytes}(X))\big) \boxplus K, \\ f_{\text{SPN}}(R, K) &= Y \lll \rho(Y), \qquad \rho(Y) = \big((5 \cdot y_0 + y_{15}) \oplus y_8\big) \bmod 128, \end{aligned} \tag{3.4}$$

dengan y_j byte ke-j dari Y dan rotasi dilakukan pada Y sebagai bilangan 128 bit *big-endian*. Alurnya ditunjukkan pada Gambar 3.3b.

3.5.1 Susunan State

Sebanyak 16 byte $X = x_0 x_1 \ldots x_{15}$ disusun menjadi matriks 4×4 **baris demi baris**. Susunan ini berbeda dengan AES yang mengisi *state* kolom demi kolom. Akibatnya, `ShiftRows` menggeser byte yang berurutan di dalam masukan, sedangkan `MixColumns` mencampur byte yang berjarak 4 posisi (lihat Tabel 3.4).

3.5.2 Kotak-S (SubBytes)

Setiap byte disubstitusi dengan Kotak-S AES (Tabel 3.3). Kotak-S ini dipilih setelah iterasi desain. Versi awal rancangan kami menggunakan Kotak-S afin $S(x) = (167x + 91) \bmod 256$, tetapi analisis pada Subbab 5.4.2 menunjukkan bahwa Kotak-S afin meloloskan selisih `0x80` dengan peluang 1. Kotak-S AES memiliki keseragaman diferensial 4 dan bias linear maksimum 16/256. Kedua nilai ini merupakan yang terbaik yang diketahui untuk permutasi 8 bit.

Tabel 3.3: Kotak-S 8×8 bit yang dipakai pada f_{SPN} (nilai heksadesimal; baris = 4 bit atas, kolom = 4 bit bawah masukan)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
A	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
B	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
C	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
D	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
E	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
F	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

3.5.3 ShiftRows dan MixColumns

`ShiftRows` merotasi baris ke-i ($i = 0, 1, 2, 3$) ke kiri sejauh i byte (Tabel 3.4). `MixColumns` mengalikan setiap kolom $(a_0, a_1, a_2, a_3)^T$ dengan matriks MDS AES di GF(2^8):

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}, \tag{3.5}$$

dengan perkalian $02 \cdot a = (a \ll 1) \oplus$ (`0x1B` jika bit 7 $a = 1$) dan $03 \cdot a = (02 \cdot a) \oplus a$. Kedua tabel perkalian ini dihitung di awal sebagai tabel *lookup*.

Tabel 3.4: Susunan *state* 4×4 sebelum dan sesudah `ShiftRows` (indeks byte masukan)

Sebelum					**Sesudah**			
x_0	x_1	x_2	x_3		x_0	x_1	x_2	x_3
x_4	x_5	x_6	x_7	$\xrightarrow{\ll 1}$	x_5	x_6	x_7	x_4
x_8	x_9	x_{10}	x_{11}	$\xrightarrow{\ll 2}$	x_{10}	x_{11}	x_8	x_9
x_{12}	x_{13}	x_{14}	x_{15}	$\xrightarrow{\ll 3}$	x_{15}	x_{12}	x_{13}	x_{14}

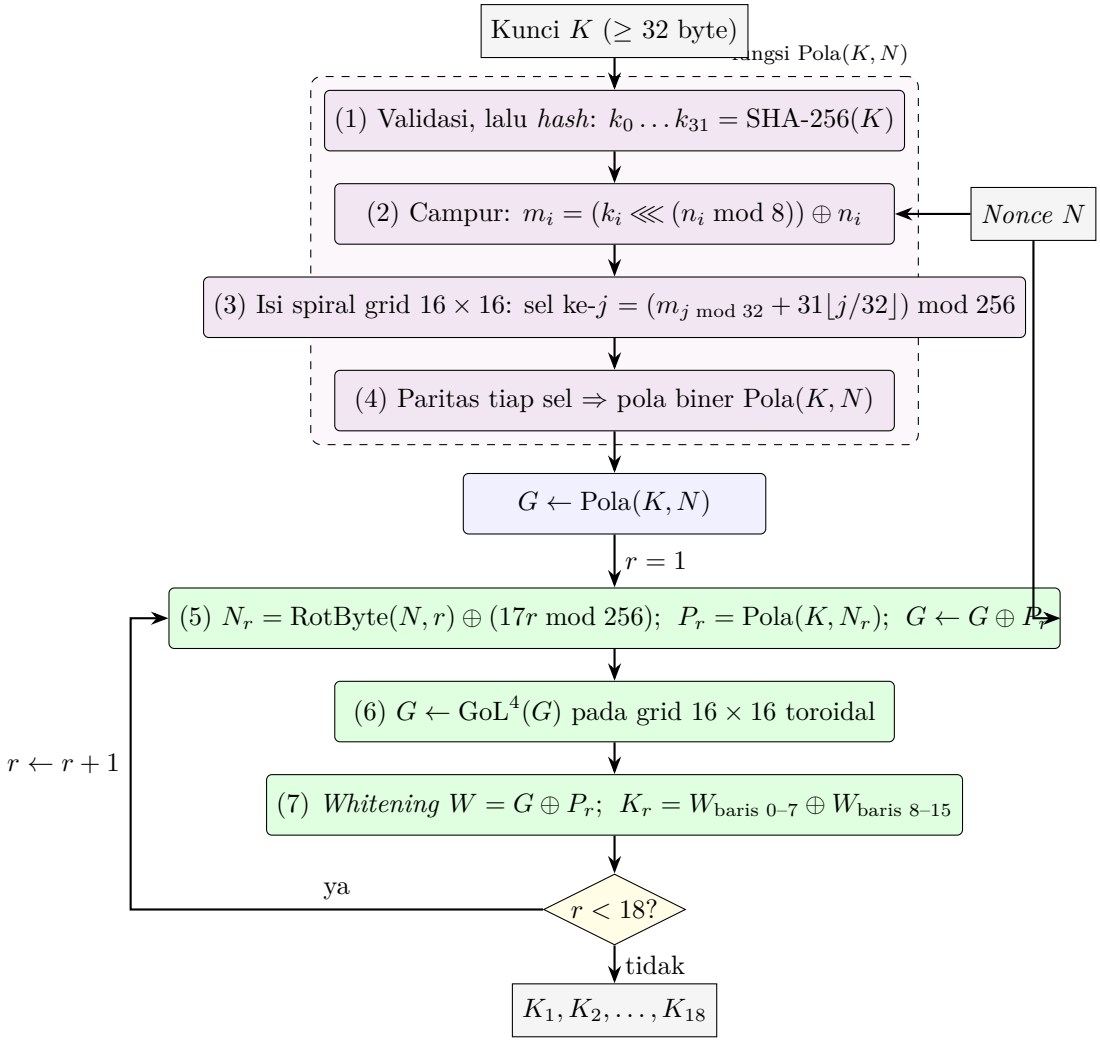
3.5.4 Penjumlahan Modular dan Rotasi Bergantung Data

Setelah `MixColumns`, *state* dijumlahkan dengan *round key* per byte modulo 256 ($\boxplus$). Kunci dengan demikian masuk dua kali ke f_{SPN} melalui dua operasi dari struktur aljabar yang berbeda: XOR (penjumlahan di GF(2)) dan penjumlahan bilangan bulat modulo 256. Pencampuran kedua operasi ini, seperti pada IDEA dan RC6 [16], mempersulit analisis yang hanya bekerja di satu struktur aljabar.

Langkah terakhir adalah rotasi kiri 128 bit sejauh $\rho(Y) \in [0, 127]$ yang dihitung dari tiga byte Y sendiri. Ide ini diambil dari RC5/RC6. Karena besar rotasi bergantung pada data, penyerang tidak dapat memprediksi posisi bit keluaran tanpa mengetahui nilai antara. Rotasi juga memindahkan bit melintasi batas byte, sehingga melengkapi `MixColumns` yang hanya bekerja di dalam kolom.

3.6 Penjadwalan Kunci (Round Key)

Penjadwalan kunci menurunkan 18 *round key* 128 bit dari kunci K dan *nonce* N. Gagasan utamanya adalah menjadikan kunci sebagai konfigurasi awal Game of Life, lalu mengambil *round key* dari hasil evolusinya setelah di-*whitening*. Alur lengkapnya ditunjukkan pada Gambar 3.5.

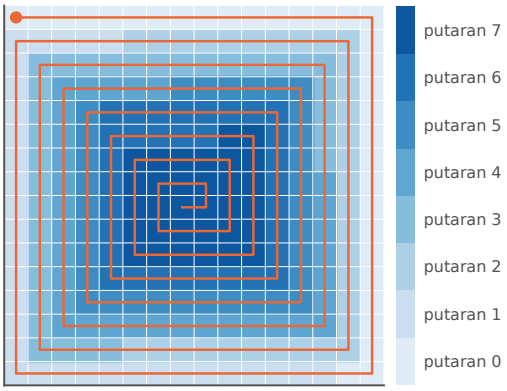


Gambar 3.5: Alur penjadwalan kunci berbasis Game of Life

Rincian setiap langkahnya adalah sebagai berikut.

1. **Validasi dan *hashing*.** Kunci minimal 32 byte (256 bit). Kunci sepanjang apa pun di-*hash* dengan SHA-256 menjadi 32 byte $k_0, \dots, k_{31}$, sehingga setiap byte kunci memengaruhi seluruh materi kunci dan tidak ada dua kunci berbeda yang (secara praktis) menghasilkan materi yang sama.
2. **Pencampuran dengan *nonce*.** Setiap byte kunci dirotasi kiri 8 bit sejauh n_i mod 8, lalu di-XOR dengan n_i (byte *nonce* ke-i). Untuk *nonce* tetap, langkah ini merupakan bijeksi pada setiap byte.
3. **Pengisian spiral.** Ke-32 byte m_i diisikan ke grid 16×16 mengikuti spiral searah jarum jam dari pojok kiri atas ke pusat (Gambar 3.6). Karena grid memiliki 256 sel, materi 32 byte berulang sebanyak 8 putaran. Pada putaran ke-ℓ ($\ell = 0, \dots, 7$), nilainya ditambah 31ℓ modulo 256 agar setiap putaran tidak sekadar menyalin putaran sebelumnya.

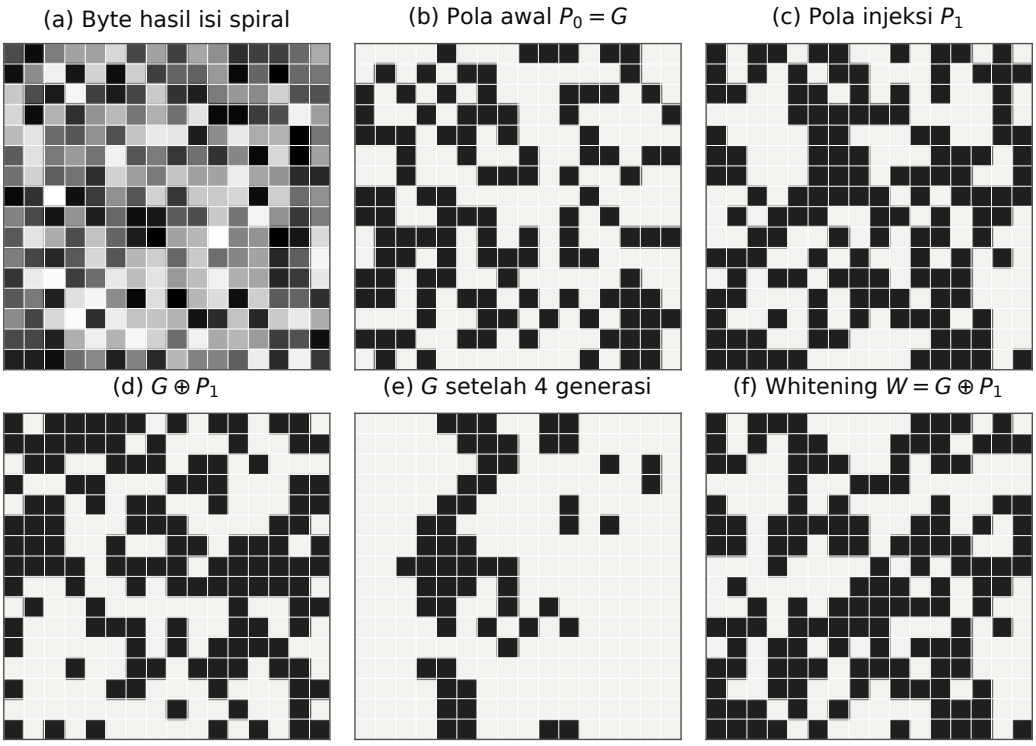
4. **Paritas.** Setiap sel diganti dengan paritas jumlah bit 1-nya (popcount mod 2), sehingga diperoleh grid biner 16×16 (256 bit) dengan kepadatan sel hidup sekitar 0,5.
5. **Injeksi ulang *nonce*.** Pada ronde ke-r, *nonce* dirotasi sejauh r byte dan setiap byte-nya di-XOR dengan $17r \bmod 256$. Pola baru $\text{Pola}(K, N_r)$ dihitung dengan langkah 1–4, lalu di-XOR ke grid. Injeksi ulang mencegah grid mati atau terjebak pada pola periodik (osilator atau *still life*), sehingga setiap ronde menerima entropi baru dari kunci.
6. **Evolusi.** Grid dievolusikan 4 generasi Game of Life pada grid toroidal 16×16.
7. ***Whitening* dan pelipatan.** Grid hasil evolusi di-XOR sekali lagi dengan pola injeksi P_r dari langkah 5, menghasilkan $W = G \oplus P_r$. Grid 256 bit W kemudian dilipat menjadi *round key* 128 bit dengan meng-XOR-kan 8 baris atas dengan 8 baris bawah. Langkah *whitening* ini menyeimbangkan proporsi bit 1 pada *round key* (Subbab 5.4.4). Grid yang dibawa ke ronde berikutnya tetap G (hasil evolusi tanpa *whitening*).



Gambar 3.6: Urutan pengisian spiral grid 16×16. Garis oranye menunjukkan urutan sel, dan warna menunjukkan putaran materi kunci ke-ℓ (32 sel per putaran).

Gambar 3.7 menunjukkan contoh nyata tahapan penjadwalan kunci untuk kunci `"kunci-rahasia-minimal-32-byte!!!"` dengan *nonce* $N = \text{SHA-256}(\texttt{"IF4020"})$. *Round key* pertama yang dihasilkan adalah

$$K_1 = \texttt{f9ec878321d2537857cb46413165e4f2}.$$



Gambar 3.7: Contoh tahapan penjadwalan kunci pada ronde pertama. Sel hitam bernilai 1 (hidup); panel (a) menampilkan nilai byte 0–255 dalam skala abu-abu. Panel (f) adalah grid yang dilipat menjadi K_1.

3.7 Dekripsi

Karena memakai struktur Feistel, dekripsi blok menggunakan prosedur yang sama dengan enkripsi, tetapi *round key* dipakai dengan urutan terbalik. Masukan C dipecah menjadi $L'_0 \| R'_0 = R_{18} \| L_{18}$, lalu untuk $j = 1, \ldots, 18$ dengan indeks kunci $i = 19 - j$:

$$L'_j = R'_{j-1}, \qquad R'_j = L'_{j-1} \oplus F_i(R'_{j-1}, K_i). \tag{3.6}$$

Jenis fungsi ronde mengikuti indeks kunci i, bukan urutan eksekusi j. Dengan demikian, ronde dekripsi pertama memakai f_{GoL} dengan K_{18}, dan ronde terakhir memakai f_{GoL} dengan K_1 (Tabel 3.5). Kebenaran dekripsi dapat ditunjukkan dengan induksi. Pada ronde pertama, $R'_1 = R_{18} \oplus F_{18}(L_{18}, K_{18}) = R_{18} \oplus F_{18}(R_{17}, K_{18}) = L_{17}$ dan $L'_1 = R_{17}$. Pola yang sama berulang hingga $R'_{18} \| L'_{18}$ bernilai $L_0 \| R_0 = P$. Pembuktian ini tidak memerlukan f_{GoL} maupun f_{SPN} untuk dapat dibalik.

Tabel 3.5: Urutan ronde pada enkripsi dan dekripsi

Urutan	**Enkripsi**		**Dekripsi**	
	Kunci	Fungsi	Kunci	Fungsi
1	K_1	f_{GoL}	K_{18}	f_{GoL}
2	K_2	f_{SPN}	K_{17}	f_{SPN}
⋮	⋮	⋮	⋮	⋮
17	K_{17}	f_{SPN}	K_2	f_{SPN}
18	K_{18}	f_{GoL}	K_1	f_{GoL}

3.8 Mode Operasi

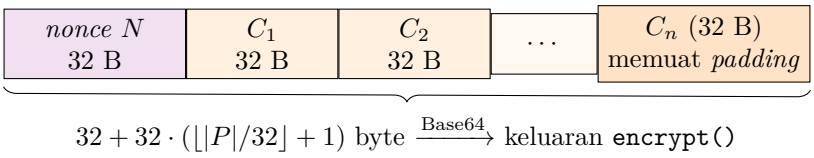
Pregnant_Sister mendukung lima mode operasi standar [4] dengan ukuran blok penuh 256 bit (Tabel 3.6). *Nonce* berperan ganda. Pada semua mode, *nonce* dipakai dalam penjadwalan kunci. Pada mode selain ECB, *nonce* juga menjadi IV atau nilai awal *counter*. Pemakaian ganda ini aman karena *nonce* dibangkitkan secara acak untuk setiap pesan, dan IV memang tidak perlu dirahasiakan.

Tabel 3.6: Mode operasi yang didukung (E/D = enkripsi/dekripsi satu blok, $C_0 = O_0 = N$)

Mode	**Enkripsi**	**Dekripsi**	**Keterangan**
ECB	$C_i = E(P_i)$	$P_i = D(C_i)$	Blok identik dalam satu pesan menghasilkan *ciphertext* identik
CBC	$C_i = E(P_i \oplus C_{i-1})$	$P_i = D(C_i) \oplus C_{i-1}$	Mode bawaan
CFB	$C_i = P_i \oplus E(C_{i-1})$	$P_i = C_i \oplus E(C_{i-1})$	CFB blok penuh (256 bit), hanya memakai E
OFB	$O_i = E(O_{i-1}),\ C_i = P_i \oplus O_i$	sama dengan enkripsi	Aliran kunci tidak bergantung pada pesan
CTR	$C_i = P_i \oplus E(N + i - 1)$	sama dengan enkripsi	N dibaca sebagai bilangan bulat 256 bit *big-endian*

3.9 Padding dan Format Ciphertext

Padding PKCS#7 diterapkan pada semua mode dengan ukuran blok 32 byte: sebanyak $p = 32 - (|P| \bmod 32)$ byte bernilai p ditambahkan, dengan $1 \le p \le 32$. Pesan yang panjangnya tepat kelipatan 32 tetap mendapat satu blok *padding* penuh (32 byte bernilai `0x20`), sehingga *padding* selalu dapat dibuang tanpa ambiguitas. Saat dekripsi, *padding* divalidasi: nilai p harus berada di rentang 1–32 dan p byte terakhir harus semuanya bernilai p. Jika tidak, dekripsi gagal dengan `ValueError`. Format keluaran akhirnya ditunjukkan pada Gambar 3.8.



Gambar 3.8: Format *ciphertext* Pregnant_Sister

Bab 4

Implementasi dan Simulasi

4.1 Lingkungan dan Struktur Program

Pregnant_Sister diimplementasikan dalam Python 3.13 tanpa pustaka kriptografi eksternal. Fungsi enkripsi dan dekripsi hanya memakai pustaka standar (`secrets` untuk *nonce*, `hashlib` untuk SHA-256 pada penjadwalan kunci, dan `base64` untuk pengodean keluaran). Pustaka `numpy` dan `matplotlib` hanya dipakai oleh modul analisis. Struktur modulnya ditunjukkan pada Tabel 4.1.

Tabel 4.1: Struktur modul pada paket `src`

Modul	**Tanggung jawab**
`encrypt.py`, `decrypt.py`	Antarmuka utama `encrypt()` dan `decrypt()`
`keygen.py`	Validasi dan *hashing* kunci, pembangkitan *nonce*, pengisian spiral, paritas, *whitening*, dan penurunan 18 *round key*
`automaton.py`	Game of Life bit-paralel pada grid toroidal (pohon penjumlah, aturan B3/S23)
`feistel.py`	Jaringan Feistel 18 ronde, pemilihan fungsi ronde, dan f_{GoL}
`spn.py`	f_{SPN}: Kotak-S, `ShiftRows`, `MixColumns`, penjumlahan modular, rotasi bergantung data
`modes.py`	Mode operasi ECB, CBC, CFB, OFB, dan CTR
`padding.py`	*Padding* PKCS#7 dan validasinya
`types.py`	Tipe data `Grid` (setiap baris disimpan sebagai *bitmask* bilangan bulat)
`analysis.py`	Uji *avalanche*, entropi, dan histogram

4.2 Antarmuka Program

Pregnant_Sister disediakan sebagai pustaka Python dengan dua fungsi utama:

- `encrypt(data, key, mode="CBC") -> bytes`: menerima *plaintext* berupa `str`, `bytes`, atau `bytearray` (teks maupun isi berkas biner) dan mengembalikan Base64 dari $N \| C$;
- `decrypt(data, key, mode="CBC") -> bytes`: menerima keluaran `encrypt()` dan mengembalikan *plaintext* asli dalam bentuk `bytes`.

Contoh pemakaian untuk teks dan berkas biner ditunjukkan pada Kode 4.1.

```python
from src.encrypt import encrypt
from src.decrypt import decrypt

key = "kunci-rahasia-minimal-32-byte!!!"            # 32 byte
ct = encrypt("Halo, dunia!", key, mode="CTR")       # bytes Base64
assert decrypt(ct, key, mode="CTR") == b"Halo, dunia!"

with open("test/berkas/image.jpg", "rb") as f:      # berkas biner
    data = f.read()
ct = encrypt(data, key, mode="OFB")
assert decrypt(ct, key, mode="OFB") == data
```

Kode 4.1: Contoh pemakaian `encrypt()` dan `decrypt()`

Pustaka ini didistribusikan melalui halaman *Releases* di GitHub. Setiap kali ada perubahan di *branch* `main`, GitHub Actions otomatis membuat rilis baru berisi arsip `pregnant_sister-vX.Y.Z.zip` (dan `.tar.gz`). Arsip ini memuat seluruh isi repositori: kode pustaka (`src`), `requirements.txt`, *test* beserta berkas ujinya, laporan, dan dokumentasi API. Pengguna cukup mengekstrak arsip, menjalankan `pip install -r requirements.txt`, lalu mengimpor `src.encrypt` dan `src.decrypt` atau menjalankan skrip pengujian dari akar arsip.

Dokumentasi API lengkap dipublikasikan di https://cola1000.github.io/Pregnant_Sister/ (Gambar 4.1). Dokumentasi ini memuat cara instalasi dari rilis, parameter dan tipe data kedua fungsi, format *ciphertext*, mode operasi, contoh penggunaan, penanganan galat, serta catatan keamanan.



Gambar 4.1: Halaman dokumentasi API Pregnant_Sister

Selain pustaka, repositori menyediakan skrip pengujian yang dijalankan dari direktori akar (Kode 4.2).

```
python -m test.test_text          # analisis berkas teks (CBC)
python -m test.test_image         # analisis berkas gambar (CBC)
python -m test.test_video         # analisis berkas video (CBC)
python -m test.test_encrypt       # round-trip lima mode untuk semua
    berkas
python -m test.test_security      # evaluasi keamanan (Bab 5)
python -m test.test_keygen        # unit test penjadwalan kunci
python -m test.test_automaton     # unit test Game of Life
```

Kode 4.2: Skrip pengujian yang tersedia

4.3 Skenario Pengujian

Pengujian dilakukan pada tiga berkas di `test/berkas` (Tabel 4.2) dengan kunci 35 byte `"000...0"` (35 karakter `'0'`). Kunci ini sengaja dipilih berpola agar terlihat bahwa *ciphertext* tetap tampak acak walaupun kuncinya lemah secara visual. Berkas teks dan citra diuji pada kelima mode, sedangkan berkas video diuji pada mode CBC. Untuk setiap pengujian, kami mencatat ukuran *ciphertext*, waktu enkripsi dan dekripsi, serta *hash* SHA-256 dari *plaintext* asli dan hasil dekripsi. Seluruh pengujian dijalankan pada mesin pengembang (Windows 11, Python 3.13.7).

Tabel 4.2: Berkas uji

Berkas	Jenis	Ukuran (byte)
`glasses.txt`	teks ASCII berbahasa Inggris	1.032
`image.jpg`	biner (citra JPEG)	833.290
`video.mp4`	biner (video MP4)	39.228.657

4.4 Hasil Enkripsi dan Dekripsi

4.4.1 Berkas Teks

Tabel 4.3 menunjukkan bahwa kelima mode mengembalikan berkas teks secara utuh. *Ciphertext* mentah berukuran 1.088 byte, yaitu 32 byte *nonce* ditambah 1.056 byte data (1.032 byte *plaintext* ditambah 24 byte *padding*). Walaupun *plaintext* dan kuncinya sama, awalan *ciphertext* berbeda pada setiap mode dan setiap eksekusi karena *nonce* dibangkitkan secara acak.

Tabel 4.3: Hasil enkripsi–dekripsi `glasses.txt` (SHA-256 *plaintext* = `76d3bcab876abc41...`)

Mode	Mentah (B)	Base64 (B)	Enk + Dek (s)	Awalan Base64	SHA-256 sama
ECB	1.088	1.452	0,02 + 0,02	`FausVPMQ6FdvselD...`	✓
CBC	1.088	1.452	0,02 + 0,02	`TMbd8jBi5LTpdZ4/...`	✓
CFB	1.088	1.452	0,02 + 0,02	`FfINU8zWIv2+NQvq...`	✓
OFB	1.088	1.452	0,02 + 0,02	`zpFhCn8xRThkoAne...`	✓
CTR	1.088	1.452	0,03 + 0,03	`s4IBPEOrgSBj4nmc...`	✓

4.4.2 Berkas Biner

Tabel 4.4 menunjukkan hasil untuk berkas citra JPEG. Kelima mode kembali mengembalikan berkas secara utuh, dan berkas hasil dekripsi dapat dibuka kembali sebagai gambar yang identik dengan aslinya. Waktu proses sekitar 12–14 detik per arah, yang setara dengan laju sekitar 60–

70 KB/s. Laju ini wajar untuk implementasi Python murni yang menjalankan 18 ronde Feistel per blok.

Tabel 4.4: Hasil enkripsi–dekripsi `image.jpg` (SHA-256 *plaintext* = `5a92b5ae1f92711d...`)

Mode	Mentah (B)	Base64 (B)	Enk + Dek (s)	Awalan Base64	SHA-256 sama
ECB	833.344	1.111.128	12,38 + 14,23	`omJYfWOjziOuq/N2...`	✓
CBC	833.344	1.111.128	11,83 + 12,07	`OZKtQ6Sq7KdFkgfE...`	✓
CFB	833.344	1.111.128	11,99 + 11,81	`ZrkGsc34edw8uSqx...`	✓
OFB	833.344	1.111.128	12,04 + 11,70	`d8268e8ICcqGvbgH...`	✓
CTR	833.344	1.111.128	11,65 + 11,96	`pW908tgQo8GVnHXq...`	✓

Berkas video 39 MB hanya diuji dengan mode CBC, karena setiap arah membutuhkan sekitar 10 menit (Tabel 4.5). Kebenaran kelima mode pada data biner sudah ditunjukkan oleh berkas citra. Skrip `test.test_encrypt` tetap dapat dipakai untuk menguji video pada kelima mode, dengan waktu total sekitar 1,5 jam.

Tabel 4.5: Hasil enkripsi–dekripsi `video.mp4` (SHA-256 *plaintext* = `f8564e7de424642c...`)

Mode	Mentah (B)	Base64 (B)	Enk + Dek (s)	Awalan Base64	SHA-256 sama
CBC	39.228.704	52.304.940	594,45 + 563,29	`3Li8Ac7JD1iAs7lM...`	✓

4.4.3 Enkripsi Probabilistik

Mengenkripsi *plaintext* `"Halo, dunia!"` dua kali dengan kunci dan mode yang sama menghasilkan dua *ciphertext* yang sama sekali berbeda:

```
sBYT/+P7ag6EmtcTZWO8guwS4HL5BWw8fa9oNjwaFlEZ...
oqGOzWvLxJ8o32eP/q0z4RkpiObsWrgvPF7z4Klit3SQ...
```

Kedua *ciphertext* tetap terdekripsi menjadi *plaintext* yang sama. Sifat ini mencegah penyerang mengetahui apakah dua pesan memiliki isi yang sama.

4.5 Analisis Penanganan Padding

Tabel 4.6 menunjukkan perilaku *padding* pada panjang *plaintext* di sekitar batas blok. Untuk setiap panjang, dilakukan *round-trip* pada kelima mode, dan semuanya berhasil.

Tabel 4.6: Perilaku *padding* PKCS#7 (blok 32 byte) pada berbagai panjang *plaintext*

$\|P\|$ (B)	p	Setelah *padding* (B)	Byte terakhir (heks)	Mentah + *nonce* (B)	*Round-trip* 5 mode
0	32	32	`20 20 20 ... 20`	64	✓
1	31	32	`1f 1f 1f ... 1f`	64	✓
15	17	32	`11 11 11 ... 11`	64	✓
31	1	32	`01`	64	✓
32	32	64	`20 20 20 ... 20`	96	✓
33	31	64	`1f 1f 1f ... 1f`	96	✓
63	1	64	`01`	96	✓
64	32	96	`20 20 20 ... 20`	128	✓
100	28	128	`1c 1c 1c ... 1c`	160	✓

Ada tiga hal penting dari hasil ini:

1. ***Padding* selalu ditambahkan.** *Plaintext* kosong dan *plaintext* yang panjangnya kelipatan 32 (0, 32, 64 byte) mendapat satu blok *padding* penuh bernilai `0x20`. Tanpa aturan ini, *plaintext* yang kebetulan berakhir dengan byte `0x01` tidak dapat dibedakan dari *padding*.
2. ***Padding* juga dipakai pada mode aliran.** CFB, OFB, dan CTR sebenarnya tidak memerlukan *padding*, tetapi rancangan kami tetap menerapkannya agar format *ciphertext* seragam untuk semua mode. Konsekuensinya, *ciphertext* selalu 1–32 byte lebih panjang dari *plaintext*.
3. ***Padding* divalidasi saat dekripsi.** Fungsi `unpad` menolak nilai p di luar rentang 1–32 dan p byte terakhir yang tidak seragam. Validasi ini juga berfungsi sebagai pendeteksi kesalahan sederhana (Subbab 4.6).

4.6 Verifikasi Integritas Data

4.6.1 Kecocokan Hash

Integritas hasil dekripsi diverifikasi dengan membandingkan *hash* SHA-256 berkas asli dan berkas hasil dekripsi. Pada Tabel 4.3 to 4.5, kesebelas pasangan berkas dan mode menghasilkan *hash* yang identik, sehingga hasil dekripsi sama persis bit demi bit dengan berkas asli. *Hash* lengkapnya dicantumkan pada Bab C. Pengujian `test_security` juga menjalankan *round-trip* untuk panjang 0, 1, 31, 32, 33, dan 100 byte dengan kunci acak sepanjang 32–71 byte pada kelima mode, dan tidak ada satu pun kegagalan.

4.6.2 Deteksi Kunci atau Mode yang Salah

Validasi *padding* memberikan deteksi kesalahan secara tidak langsung. Jika kunci salah, blok terakhir hasil dekripsi praktis acak, sehingga peluang *padding*-nya kebetulan valid sekitar $\sum_{p=1}^{32} 2^{-8p} \approx 1/255$. Hasil eksperimen pada Tabel 4.7 sesuai dengan perkiraan tersebut.

Tabel 4.7: Deteksi dekripsi dengan kunci atau mode yang salah (400 percobaan per skenario)

Skenario	**Terdeteksi (`ValueError`)**	**Lolos tanpa galat**
Kunci salah, mode CBC	399	1
Kunci salah, mode CTR	397	3
Enkripsi CBC, dekripsi CTR	399	1

4.6.3 Deteksi Perubahan Ciphertext

Untuk menguji apakah perubahan pada *ciphertext* terdeteksi, kami membalik satu bit pada blok kedua *ciphertext* sebuah pesan empat blok, lalu menghitung jumlah bit yang berubah pada setiap blok hasil dekripsi (Tabel 4.8). Kami juga membalik satu bit acak pada blok terakhir sebanyak 300 kali per mode.

Tabel 4.8: Propagasi galat akibat pembalikan 1 bit pada blok *ciphertext* C_2 (jumlah bit berubah dari 256 bit per blok *plaintext*)

Mode	P_1	P_2	P_3	P_4	P_5 (*pad*)	**Terdeteksi?**	**Ubah blok terakhir terdeteksi**
ECB	0	120	0	0	0	tidak	299/300
CBC	0	130	1	0	0	tidak	298/300
CFB	0	1	126	0	0	tidak	133/300
OFB	0	1	0	0	0	tidak	125/300
CTR	0	1	0	0	0	tidak	131/300

Hasil ini sesuai dengan teori mode operasi. Pada ECB dan CBC, blok yang diubah terdekripsi menjadi data acak, dan pada CBC satu bit yang sama juga ikut terbalik di blok berikutnya. Pada CFB terjadi kebalikannya: bit yang sama terbalik di blok tersebut, lalu blok berikutnya rusak. Pada OFB dan CTR, hanya bit yang sama di *plaintext* yang terbalik. **Tidak satu pun perubahan di tengah pesan terdeteksi**, karena blok *padding* tidak tersentuh.

Perubahan pada blok terakhir terdeteksi hampir selalu pada ECB/CBC (seluruh blok terakhir menjadi acak sehingga *padding* rusak). Pada mode aliran, perubahan hanya terdeteksi jika bit yang dibalik jatuh di area *padding* (14 dari 32 byte pada pesan ini, sekitar 44%). Kesimpulannya, **validasi *padding* bukan mekanisme integritas**. Pregnant_Sister memberikan kerahasiaan, tetapi tidak memberikan autentikasi pesan. Implikasi keamanannya dibahas pada Subbab 5.4.6.

Bab 5

Analisis Keamanan

Bab ini membahas hasil pengujian *avalanche effect*, entropi, dan histogram, lalu dilanjutkan dengan pengujian tambahan terhadap komponen rancangan. Pengujian tingkat blok dijalankan dengan `test/test_security.py`, yang keluaran lengkapnya terdapat pada Bab B. Pengujian tingkat berkas memakai berkas dan kunci yang sama dengan Bab 4.

5.1 Avalanche Effect

5.1.1 Tingkat Blok

Uji *avalanche* paling bersih dilakukan pada satu blok dengan *round key* tetap, agar pengaruh *nonce* acak tidak ikut terhitung. Untuk 20 blok acak, setiap bit dari 256 bit *plaintext* dibalik satu per satu (total 5.120 pasangan), lalu dihitung jarak Hamming antara kedua *ciphertext*. Hal yang sama dilakukan untuk setiap bit kunci ke-4 (64 pasangan) dengan *nonce* tetap. Hasilnya ditunjukkan pada Tabel 5.1.

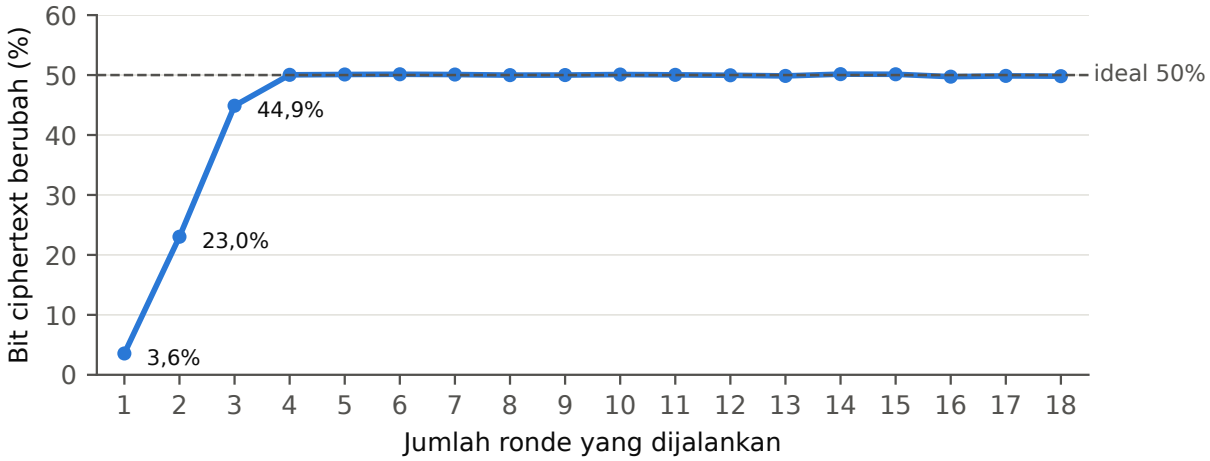
Tabel 5.1: *Avalanche effect* tingkat blok (256 bit)

Perubahan 1 bit pada	**Rata-rata**	**Minimum**	**Maksimum**
Plaintext	49,95%	98 bit (38,3%)	162 bit (63,3%)
Kunci	50,46%	115 bit (44,9%)	–

Rata-rata keduanya sangat dekat dengan nilai ideal 50%. Rentang 98–162 bit juga konsisten dengan distribusi binomial $\mathrm{B}(256, \frac{1}{2})$, yang memiliki simpangan baku 8 bit. Nilai ekstrem 98 dan 162 bit berjarak sekitar 3,8 dan 4,3 simpangan baku dari rata-rata 128. Dengan 5.120 sampel, peluang munculnya minimal satu nilai sejauh 4,3 simpangan baku secara kebetulan sekitar 10%, sehingga hasil ini sesuai dengan perilaku permutasi acak.

5.1.2 Difusi terhadap Jumlah Ronde

Untuk melihat seberapa cepat difusi tercapai, *avalanche* diukur ulang dengan hanya menjalankan n ronde pertama, untuk $n = 1, \ldots, 18$ (Gambar 5.1). Difusi penuh (sekitar 50%) tercapai setelah **4 ronde**. Hal ini sesuai dengan struktur Feistel: satu bit di R_0 memengaruhi seluruh R_1 melalui F, lalu membutuhkan satu ronde lagi untuk menyebar ke paruh lainnya. Dengan 18 ronde, Pregnant_Sister memiliki margin keamanan sekitar 4,5 kali jumlah ronde minimum untuk difusi penuh. Margin ini sebanding dengan AES-128, yang mencapai difusi penuh setelah 2 ronde dan memakai 10 ronde.



Gambar 5.1: Persentase bit *ciphertext* yang berubah terhadap jumlah ronde yang dijalankan (12 blok × 64 posisi bit per titik)

5.1.3 Tingkat Berkas pada Setiap Mode

Tabel 5.2 menunjukkan *avalanche* pada 2.048 byte pertama setiap berkas (seluruh isi untuk `glasses.txt`). *Nonce* yang sama dipakai untuk kedua enkripsi, dan bit yang dibalik adalah bit pertama *plaintext* atau bit terakhir byte pertama kunci.

Tabel 5.2: *Avalanche effect* tingkat berkas per mode operasi

Mode	1 bit ***plaintext***		1 bit kunci	
	`glasses.txt`	`image.jpg`	`glasses.txt`	`image.jpg`
ECB	1,46%	0,72%	50,27%	50,67%
CBC	49,89%	49,87%	49,47%	50,52%
CFB	48,88%	49,18%	50,39%	50,57%
OFB	0,01%	0,01%	50,66%	50,79%
CTR	0,01%	0,01%	50,79%	49,95%

Perubahan **kunci** menghasilkan *avalanche* sekitar 50% pada kelima mode, karena seluruh *round key* ikut berubah. Sebaliknya, *avalanche* terhadap perubahan ***plaintext*** sangat bergantung pada mode:

- **CBC dan CFB** (sekitar 50%): perubahan pada blok pertama merambat ke semua blok berikutnya melalui rantai umpan balik.
- **ECB** (1,46% dan 0,72%): hanya blok pertama yang berubah, dan sekitar setengah bitnya berubah. Karena blok pertama hanya 1/33 dan 1/65 bagian dari data yang diukur (termasuk blok *padding*), persentase totalnya kecil.
- **OFB dan CTR** (0,01% = tepat 1 bit): aliran kunci tidak bergantung pada *plaintext*, sehingga membalik satu bit *plaintext* hanya membalik satu bit yang sama di *ciphertext*.

Nilai kecil pada ECB, OFB, dan CTR **bukan kelemahan *block cipher***, melainkan sifat bawaan mode tersebut yang juga berlaku pada AES. Hasil tingkat blok pada Tabel 5.1 menunjukkan bahwa fungsi E sendiri memiliki *avalanche* yang ideal. Hal ini juga menjelaskan mengapa `src/analysis.py` memakai mode CBC untuk pengukuran *avalanche*.

5.2 Entropi

Entropi Shannon dihitung pada *ciphertext* mentah tanpa *nonce*. Base64 tidak dipakai karena hanya memiliki 64 simbol, sehingga entropinya tidak mungkin melebihi 6 bit per byte. Hasilnya ditunjukkan pada Tabel 5.3.

Tabel 5.3: Entropi Shannon (bit per byte, maksimum 8) *plaintext* dan *ciphertext*

Berkas	***Plaintext***	***Ciphertext***					**Harapan acak**
		ECB	CBC	CFB	OFB	CTR	
`glasses.txt`	4,3935	7,7762	7,8292	7,8137	7,8386	7,8077	7,8258
`image.jpg`	7,9306	7,9997	7,9998	7,9998	7,9998	7,9998	7,9998
`video.mp4`	7,9993	–	8,0000	–	–	–	8,0000
64 KB byte nol	0,0000	–	7,9973	–	–	–	7,9972

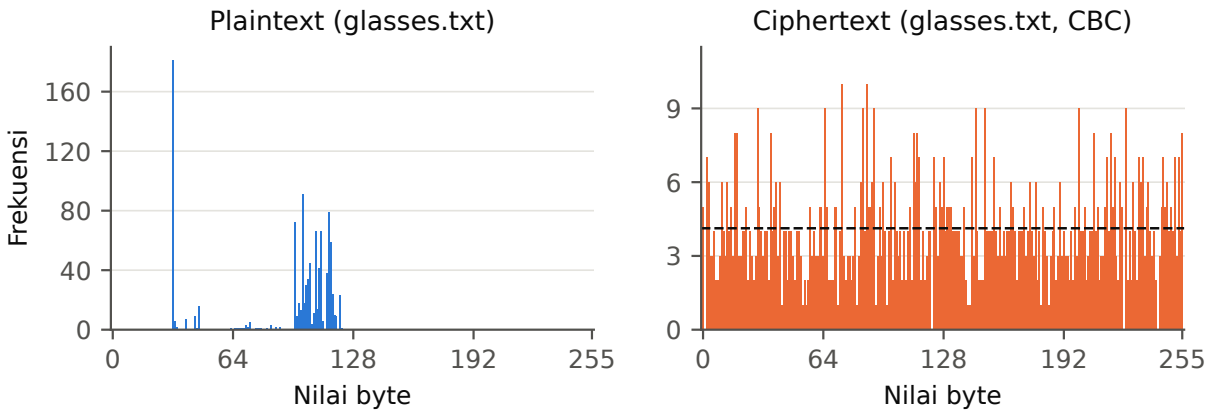
Entropi *ciphertext* `glasses.txt` (7,78–7,84) tampak lebih rendah daripada berkas lain, tetapi nilai ini memang yang diharapkan dari data acak sempurna berukuran sama. Entropi yang diukur dari sampel berhingga sebanyak n byte bersifat bias ke bawah, dengan nilai harapan sekitar [11]

$$\mathbb{E}[\hat{H}] \approx 8 - \frac{255}{2n \ln 2}. \tag{5.1}$$

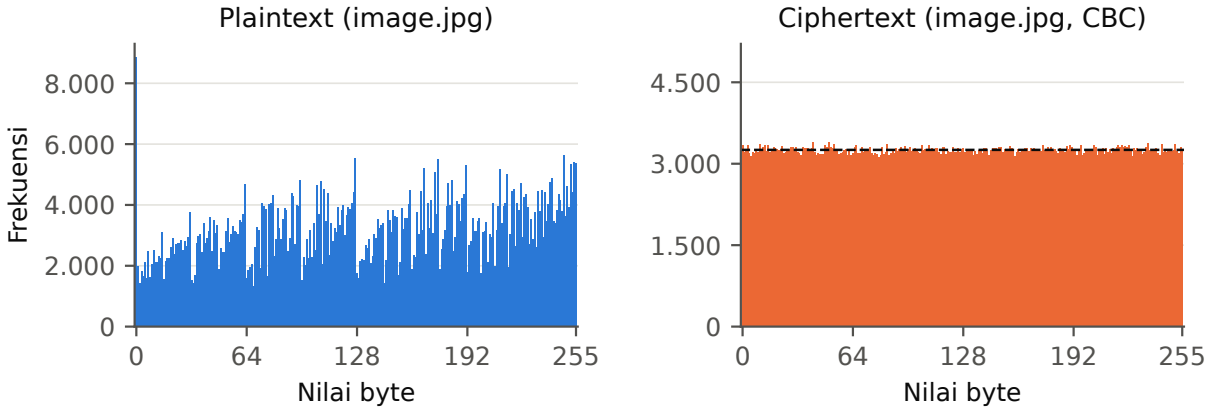
Untuk $n = 1.056$ byte, nilai harapannya 7,8258, sehingga hasil 7,78–7,84 sesuai dengan keluaran yang acak. Untuk citra ($n = 833.312$), nilai harapannya 7,9998, dan untuk video ($n = 39.228.672$) nilai harapannya 7,999995. Keduanya sama dengan hasil pengukuran. Video MP4 yang sudah terkompresi memiliki entropi *plaintext* 7,9993, sehingga entropi saja tidak dapat membedakan *plaintext* terkompresi dari *ciphertext*. Karena itu, histogram dan uji χ^2 pada subbab berikutnya tetap diperlukan. Kasus paling ekstrem adalah 64 KB byte nol (entropi *plaintext* 0): *ciphertext*-nya tetap mencapai entropi 7,9973. Artinya, keacakan keluaran tidak bergantung pada keacakan masukan.

5.3 Histogram

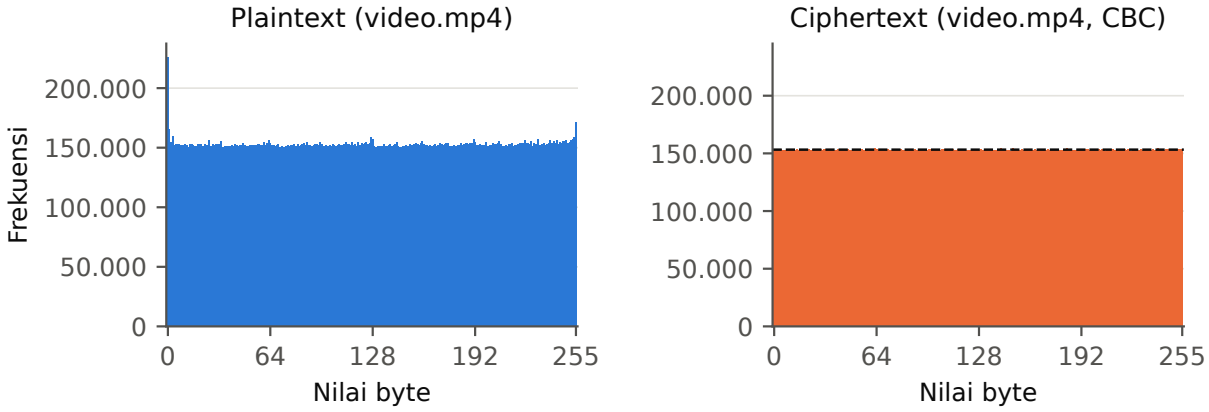
Gambar 5.2 to 5.4 membandingkan histogram nilai byte *plaintext* dan *ciphertext* (mode CBC). Histogram *plaintext* teks terkonsentrasi pada rentang huruf ASCII (sekitar 97–122) dan spasi (32). Histogram citra JPEG tidak rata, dengan puncak paling menonjol di nilai `0x00` (sekitar 2,7 kali rata-rata). Setelah dienkripsi, kedua histogram menjadi rata di sekitar nilai harapan (garis putus-putus), tanpa sisa pola dari *plaintext*.



Gambar 5.2: Histogram `glasses.txt`: *plaintext* (kiri) dan *ciphertext* CBC (kanan). Garis putus-putus menunjukkan frekuensi harapan distribusi seragam.



Gambar 5.3: Histogram `image.jpg`: *plaintext* (kiri) dan *ciphertext* CBC (kanan)



Gambar 5.4: Histogram `video.mp4`: *plaintext* (kiri) dan *ciphertext* CBC (kanan)

Kerataan histogram diuji secara kuantitatif dengan uji χ^2 (255 derajat kebebasan, nilai kritis 293,25 pada $\alpha = 5\%$). Hasilnya ditunjukkan pada Tabel 5.4.

Tabel 5.4: Statistik χ^2 histogram (lolos jika < 293,25)

Berkas	***Plaintext***	**ECB**	**CBC**	**CFB**	**OFB**	**CTR**
`glasses.txt`	17.551	309,3*	241,5	257,0	224,5	264,2
`image.jpg`	81.473	304,1*	228,4	250,7	262,4	252,9
`video.mp4`	42.092	–	252,4	–	–	–
64 KB byte nol	–	–	239,6	–	–	–

Seluruh *ciphertext* pada mode CBC, CFB, OFB, dan CTR, serta *ciphertext* video dan berkas nol, lolos uji χ^2. Dua nilai pada mode ECB melewati batas: 309,3 pada berkas teks ($p \approx 0{,}011$) dan 304,1 pada citra ($p \approx 0{,}019$). ECB memang memetakan blok *plaintext* identik ke blok *ciphertext* identik, sehingga kami terlebih dahulu memeriksa apakah blok berulang menjadi penyebabnya. Ternyata `glasses.txt` tidak memiliki blok 32 byte yang berulang, dan `image.jpg` hanya memiliki 33 blok duplikat dari 26.040 blok, jumlah yang terlalu kecil untuk memengaruhi histogram. Karena itu, kami mengulang enkripsi berkas teks sebanyak 300 kali dengan *nonce* berbeda untuk setiap mode, dan mengulang enkripsi ECB citra sebanyak 20 kali (Tabel 5.5).

Tabel 5.5: Distribusi χ^2 dari enkripsi ulang dengan *nonce* berbeda

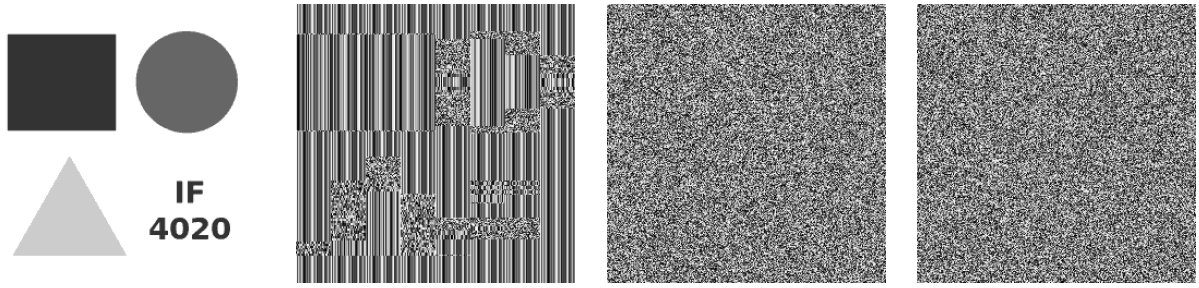
Mode	**Rata-rata**	**Simpangan baku**	**Melewati 293,25**	**Maksimum**
`glasses.txt`, 300 kali per mode				
ECB	255,8	22,8	7,0%	314,7
CBC	254,6	21,9	3,7%	331,6
CFB	255,1	21,5	5,3%	333,1
OFB	256,2	23,0	6,0%	323,4
CTR	255,1	21,4	3,0%	341,8
`image.jpg`, 20 kali				
ECB	253,8	24,8	15% (3 dari 20)	300,7
Teoretis χ^2_{255}	255,0	22,6	5,0%	–

Pada kelima mode, rata-rata dan simpangan baku χ^2 sesuai dengan distribusi teoretis χ^2_{255}. Proporsi yang melewati batas (3,0–7,0%) berada dalam sekitar 1,6 galat baku dari nilai nominal 5% (galat baku ±1,3% untuk 300 percobaan). Pada citra, 3 dari 20 percobaan ECB melewati batas, tetapi peluang mendapatkan minimal 3 kegagalan dari 20 percobaan secara kebetulan sekitar 7,5%, dan rata-ratanya (253,8) tetap sesuai harapan. Enkripsi CTR atas 1 MB byte nol juga menghasilkan $\chi^2 = 238{,}8$. Dengan demikian, kegagalan pada Tabel 5.4 merupakan fluktuasi acak, bukan bias pada mode tertentu.

5.4 Pengujian Tambahan

5.4.1 Kebocoran Pola pada Mode ECB

Kelemahan klasik ECB diuji dengan citra mentah berukuran 256×256 piksel *grayscale* (1 byte per piksel) yang memiliki banyak area seragam (Gambar 5.5). Citra ini memiliki 2.048 blok, tetapi hanya 270 blok yang berbeda. Pada ECB, *ciphertext*-nya juga memiliki tepat 270 blok berbeda, sehingga bentuk-bentuk pada citra masih terlihat jelas. Pada CBC dan CTR, ke-2.048 blok *ciphertext* semuanya berbeda dan citra tampak seperti derau. Percobaan dengan 100 blok *plaintext* identik menunjukkan hal yang sama: ECB hanya menghasilkan 2 blok *ciphertext* berbeda, yaitu blok data dan blok *padding*.



(a) *Plaintext* (b) ECB (c) CBC (d) CTR

Gambar 5.5: Citra mentah yang dienkripsi dengan ECB, CBC, dan CTR (setiap baris citra 256 piksel = 8 blok)

Nonce per pesan membuat Pregnant_Sister lebih baik daripada ECB pada AES dalam satu hal: blok yang sama pada **dua pesan berbeda** menghasilkan *ciphertext* berbeda, karena *round key*-nya berbeda. Namun, di **dalam satu pesan**, ECB tetap membocorkan pola, sehingga mode ini tidak disarankan untuk data berpola.

5.4.2 Analisis Kotak-S dan Iterasi Desain

Versi awal f_{SPN} memakai Kotak-S afin $S(x) = (167x + 91) \bmod 256$. Analisis diferensial menunjukkan bahwa selisih $\Delta = \texttt{0x80}$ pada setiap byte lolos melalui seluruh komponen f_{SPN}:

- **Kotak-S**: $x \oplus \texttt{0x80} = x + 128 \pmod{256}$, sehingga $S(x \oplus \texttt{0x80}) = S(x) + 167 \cdot 128 = S(x) + 128 = S(x) \oplus \texttt{0x80}$ (karena 167 ganjil). Peluangnya 1.
- **`MixColumns`**: jika keempat byte suatu kolom memiliki selisih `0x80`, selisih keluarannya adalah $(02 \oplus 03 \oplus 01 \oplus 01) \cdot \texttt{0x80} = \texttt{0x80}$. Peluangnya 1.
- **Penjumlahan mod 256**: selisih `0x80` hanya mengenai bit paling signifikan, sehingga tidak menghasilkan *carry*. Peluangnya 1.
- **Rotasi**: besar rotasi ρ tidak berubah, karena bit 7 dihapus oleh mod 128, dan pola `0x80` berulang tetap terjaga jika $\rho \equiv 0 \pmod 8$. Peluangnya 2^{-3}.

Akibatnya, setiap ronde SPN yang aktif hanya memberikan peluang 2^{-3}. Ini adalah kelemahan fatal. Kotak-S kemudian diganti dengan Kotak-S AES. Tabel 5.6 membandingkan kedua Kotak-S, dan pengujian ulang menunjukkan bahwa karakteristik `0x80` tidak pernah muncul lagi (0 dari 4.000 percobaan pada f_{SPN} dan 0 dari 20.000 percobaan pada 6 ronde tereduksi).

Tabel 5.6: Perbandingan Kotak-S versi awal dan versi akhir

Sifat	**Afin** $(167x + 91) \bmod 256$	**AES (versi akhir)**
Afin terhadap $\mathbb{Z}_{256}$	ya	tidak
Nilai maksimum DDT	256/256 (untuk $\Delta = \texttt{0x80}$)	4/256
Bias linear maksimum (LAT)	128/256 (bit terendah linear)	16/256
Peluang `0x80..80` $\rightarrow$ `0x80..80` pada f_{SPN}	2^{-3}	0/4.000 (tidak teramati)

5.4.3 Batas Kriptanalisis Diferensial

Pada jaringan Feistel dengan fungsi ronde yang (hampir) bijektif, setiap tiga ronde berurutan memiliki minimal dua ronde aktif. Jika ronde i tidak aktif ($\Delta R_{i-1} = 0$), maka $\Delta R_i = \Delta L_{i-1} \neq 0$ menjadi masukan ronde $i + 1$, sedangkan $\Delta L_i = 0$. Masukan ronde $i + 2$ adalah $\Delta R_{i+1} =$

$\Delta L_i \oplus \Delta F_{i+1} = \Delta F_{i+1}$, yang tidak nol jika F bijektif. Dari 16 ronde SPN, minimal 10 ronde aktif. Setiap f_{SPN} yang aktif mengaktifkan minimal satu Kotak-S dengan peluang diferensial maksimum 2^{-6}, sehingga peluang karakteristik diferensial apa pun dibatasi oleh

$$p \leq \left(2^{-6}\right)^{10} = 2^{-60}. \tag{5.2}$$

Batas ini dapat dibuktikan, tetapi masih longgar. Batas ini belum mencapai 2^{-256} (ukuran blok), sehingga belum menjadi jaminan formal. Batas sebenarnya hampir pasti jauh lebih kecil, karena `MixColumns` dengan *branch number* 5 umumnya mengaktifkan banyak Kotak-S pada ronde berikutnya, dan rotasi yang bergantung data mengacak posisi selisih. Skrip `test_security` memberikan estimasi heuristik $\leq 2^{-132}$ untuk karakteristik iteratif 3 ronde, tetapi estimasi ini hanya berlaku untuk satu jenis karakteristik. Pembuktian batas yang ketat, misalnya dengan menghitung jumlah minimum Kotak-S aktif menggunakan MILP, menjadi salah satu saran pengembangan.

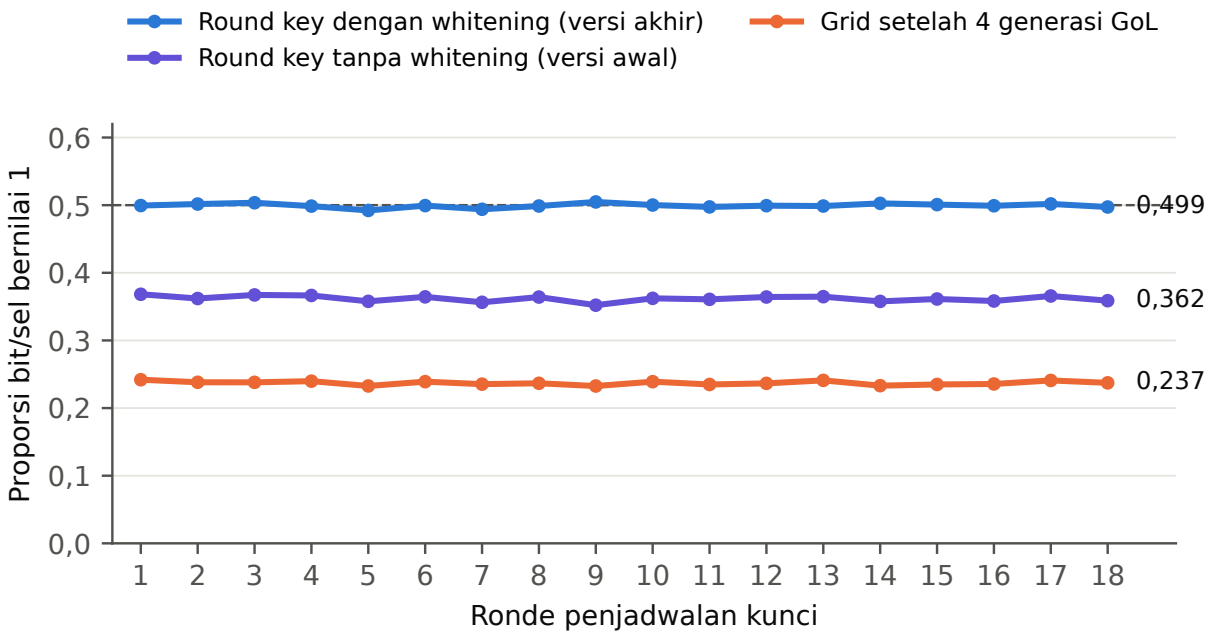
5.4.4 Iterasi Desain Penjadwalan Kunci: Distribusi Bit Round Key

Versi awal penjadwalan kunci mengambil *round key* langsung dari lipatan grid hasil evolusi, yaitu $K_r = \text{Lipat}(G)$. Pengukuran pada 300 pasangan kunci dan *nonce* acak menunjukkan bahwa proporsi bit 1 pada *round key* versi tersebut hanya **0,362**, bukan 0,5. Penyebabnya dapat dijelaskan dengan mengukur kepadatan grid pada setiap tahap:

1. Setelah injeksi ulang, kepadatan grid sekitar 0,500, karena XOR dengan pola paritas yang seimbang mengembalikan grid ke keseimbangan.
2. Empat generasi GoL menurunkan kepadatan menjadi sekitar $q = 0{,}237$, karena sel yang terlalu padat mati akibat *overpopulation*. Penurunan ini adalah perilaku umum GoL pada konfigurasi acak.
3. Pelipatan dengan XOR dua bit yang masing-masing bernilai 1 dengan peluang q menghasilkan bit 1 dengan peluang $2q(1-q) = 2 \cdot 0{,}237 \cdot 0{,}763 = 0{,}362$, sesuai dengan hasil pengukuran.

Jika bit-bitnya dianggap independen, entropi per bit *round key* versi awal hanya $H(0{,}362) \approx 0{,}944$ bit, atau sekitar 120,9 bit per *round key* 128 bit.

Versi akhir memperbaikinya dengan *whitening*: sebelum dilipat, grid hasil evolusi di-XOR dengan pola injeksi P_r dari ronde yang sama, sehingga $K_r = \text{Lipat}(G \oplus P_r)$. Pola P_r memiliki kepadatan sekitar 0,5, sehingga XOR dengannya menghapus bias yang ditimbulkan GoL. Grid yang dibawa ke ronde berikutnya tetap G, jadi dinamika GoL pada penjadwalan kunci tidak berubah; yang berubah hanya cara *round key* diekstraksi. Gambar 5.6 membandingkan kedua versi.



Gambar 5.6: Proporsi bit 1 pada grid dan *round key* untuk setiap ronde penjadwalan kunci (300 pasangan kunci dan *nonce* acak; angka di kanan = rata-rata 18 ronde; garis putus-putus = 0,5)

Setelah *whitening*, proporsi bit 1 menjadi **0,499**. Pengujian yang lebih rinci pada 54.000 *round key* (3.000 pasangan kunci dan *nonce* acak) menunjukkan bahwa masih tersisa bias kecil yang bergantung pada posisi bit. Proporsi per posisi berada di rentang 0,471–0,515, dan 3 dari 128 posisi menyimpang secara signifikan dari 0,5 ($|z| > 4$). Bit pada *round key* yang berurutan tidak berkorelasi (koefisien korelasi 0,0001), sehingga bias ini bukan artefak sampel. Penyebabnya kemungkinan struktur spasial pengisian spiral, karena delapan sel yang berasal dari byte materi kunci yang sama saling berkorelasi. Dampaknya sangat kecil: jika setiap posisi dianggap independen, entropi *round key* hanya berkurang sekitar 0,007 bit dari 128 bit. Entropi byte *round key* juga mencapai 7,9981 dari 86.400 byte, sesuai dengan nilai harapan 7,9979 untuk data acak berukuran sama. Uji `test_security` versi akhir juga melaporkan proporsi 0,4985 dan entropi byte 7,9934 (harapan 7,9936 untuk 28.800 byte) (Bab B).

Kami juga memeriksa apakah langkah paritas menghilangkan informasi kunci. Pada satu pola, setiap byte materi kunci (keluaran SHA-256 yang telah dicampur dengan *nonce*) menentukan 8 bit paritas (satu untuk setiap putaran spiral), tetapi hanya ada 98 kombinasi yang berbeda dari 256 kemungkinan nilai byte (sekitar 6,3 bit informasi). Namun, jika ke-19 pola (P_0 dan 18 injeksi ulang dengan *nonce* berbeda) digabungkan, ke-256 nilai setiap byte materi kunci dapat dibedakan pada 50 *nonce* acak yang diuji. Dengan demikian, langkah paritas tidak menyempitkan ruang kunci di tingkat pola.

5.4.5 Iterasi Desain Penjadwalan Kunci: Kunci Ekuivalen

Versi awal melipat kunci yang lebih panjang dari 32 byte dengan XOR, sehingga kunci yang berbeda dapat menghasilkan materi kunci yang sama. Contoh ekstremnya, setiap kunci berbentuk $X \| X$ terlipat menjadi 32 byte nol. Pada versi tersebut, *ciphertext* yang dienkripsi dengan kunci `"my secret key that is 32 bytes!!"` ×2 dapat didekripsi dengan kunci `"A"` ×64.

Versi akhir mengganti pelipatan dengan SHA-256. Karena SHA-256 tahan tabrakan, dua kunci berbeda praktis tidak mungkin menghasilkan materi kunci yang sama, dan setiap bit kunci memengaruhi seluruh 256 bit materi kunci. Pada versi akhir, dekripsi dengan kunci `"A"` ×64 gagal dengan `ValueError` (*padding* tidak valid), dan `test_security` melaporkan bahwa kedua

kunci tidak lagi saling menggantikan. Perlu dicatat bahwa SHA-256 adalah fungsi *hash* yang cepat, bukan KDF untuk kata sandi. Jika kunci berasal dari frasa sandi berentropi rendah, KDF lambat dengan *salt* tetap lebih tepat (Subbab 6.2).

5.4.6 Integritas dan Malleability

Seperti ditunjukkan pada Tabel 4.8, Pregnant_Sister tidak memiliki autentikasi pesan. Penyerang yang mengetahui sebagian isi *plaintext* dapat mengubahnya secara terarah tanpa mengetahui kunci. Pada percobaan dengan mode CBC, membalik bit yang tepat di blok *ciphertext* pertama mengubah *plaintext* blok kedua dari `"PAY 0000100 USD..."` menjadi `"PAY 9000100 USD..."`, dan dekripsi tetap berhasil tanpa galat. Pada OFB dan CTR, manipulasi ini bahkan lebih mudah karena tidak ada blok yang rusak. Selain itu, fungsi `unpad` mengeluarkan dua pesan galat yang berbeda (`Invalid padding length` dan `Invalid padding sequence`). Jika pesan galat ini terlihat oleh penyerang, ia dapat dimanfaatkan sebagai *padding oracle* pada mode CBC [20]. Kedua masalah ini diatasi dengan skema *Encrypt-then-MAC*, misalnya HMAC-SHA256 atas $N \| C$ dengan kunci terpisah [1, 9].

5.4.7 Kinerja

Kinerja diukur pada mesin pengembang dengan Python 3.13 (Tabel 5.7). Penjadwalan kunci hanya dijalankan sekali per pesan, sehingga biaya utamanya adalah 18 ronde Feistel per blok. Laju sekitar 60–70 KB/s membuat implementasi ini cukup untuk berkas berukuran hingga beberapa MB, tetapi berkas 39 MB sudah membutuhkan hampir 10 menit per arah. Kinerja yang rendah ini sebagian besar disebabkan oleh interpretasi Python, bukan oleh rancangan algoritma. Operasi GoL bit-paralel, Kotak-S, dan `MixColumns` pada dasarnya hanya terdiri atas operasi bitwise dan *lookup* tabel yang cepat di perangkat keras.

Tabel 5.7: Hasil pengukuran kinerja

Operasi	**Waktu / laju**
Penjadwalan kunci (18 *round key*)	5,7 ms per pesan
Enkripsi satu blok (32 byte)	474 μs ($\approx$ 66 KB/s)
Enkripsi / dekripsi `image.jpg` (833 KB)	$\approx$ 12 s ($\approx$ 68 KB/s)
Enkripsi / dekripsi `video.mp4` (39,2 MB, CBC)	594 s / 563 s ($\approx$ 64–68 KB/s)

5.5 Ringkasan Hasil Analisis

Tabel 5.8: Ringkasan hasil analisis keamanan

Aspek	**Hasil**	**Status**
Round-trip	teks dan citra pada 5 mode, video pada CBC, 9 panjang *padding* pada 5 mode; *hash* identik	baik
Avalanche plaintext/kunci	49,95% / 50,46% pada tingkat blok	baik
Difusi	penuh setelah 4 dari 18 ronde	baik
Entropi	sesuai harapan data acak pada semua ukuran	baik
Histogram	rata; kegagalan χ^2 yang sesekali muncul terbukti fluktuasi acak melalui pengujian berulang	baik
Kotak-S	DDT 4/256, LAT 16/256 (setelah iterasi desain)	baik
Batas diferensial terbukti	$\leq 2^{-60}$ (longgar)	perlu kajian
Bit *round key*	proporsi 0,499 setelah *whitening* (sebelumnya 0,362); bias kecil per posisi, kehilangan entropi $\approx$ 0,007 bit	baik
Derivasi kunci	SHA-256, tidak ada lagi kunci ekuivalen $X\|\|X$	baik
ECB	membocorkan pola di dalam satu pesan	sifat mode
Integritas	tidak ada MAC; *ciphertext* dapat dimanipulasi	lemah

Bab 6

Kesimpulan dan Saran Pengembangan

6.1 Kesimpulan

1. Kami telah merancang Pregnant_Sister, sebuah *block cipher* 256 bit berstruktur Feistel 18 ronde yang memadukan Conway's Game of Life dengan komponen SPN gaya AES. GoL dipakai di dua tempat: fungsi ronde f_{GoL} pada ronde pertama dan terakhir, serta penjadwalan kunci yang menurunkan 18 *round key* dari evolusi grid 16×16 dengan injeksi ulang *nonce* dan *whitening*. Struktur Feistel memungkinkan GoL, yang tidak dapat dibalik, dipakai sebagai fungsi ronde tanpa mengganggu proses dekripsi.

2. Implementasi Python berhasil mengenkripsi dan mendekripsi berkas teks maupun biner pada kelima mode operasi (ECB, CBC, CFB, OFB, CTR). Seluruh hasil dekripsi identik bit demi bit dengan berkas asli, dibuktikan dengan kecocokan *hash* SHA-256. *Padding* PKCS#7 menangani semua panjang pesan dengan benar, termasuk pesan kosong dan pesan yang panjangnya kelipatan ukuran blok.

3. Secara statistik, Pregnant_Sister berperilaku seperti permutasi acak. *Avalanche* terhadap *plaintext* dan kunci masing-masing 49,95

4. Analisis komponen menghasilkan tiga iterasi desain. Kotak-S afin pada versi awal meloloskan selisih `0x80` dengan peluang 1, sehingga diganti dengan Kotak-S AES. Pelipatan kunci dengan XOR membuat kunci $X \| X$ setara dengan kunci nol, sehingga diganti dengan SHA-256. *Round key* yang diambil langsung dari grid hasil evolusi hanya memiliki proporsi bit 1 sebesar 0,362, sehingga ditambahkan *whitening* yang menaikkannya menjadi 0,499. Temuan-temuan ini menunjukkan bahwa uji statistik seperti *avalanche* dan entropi tidak cukup untuk menilai keamanan, karena versi awal pun lolos uji-uji tersebut.

5. Beberapa keterbatasan masih tersisa pada versi akhir: *ciphertext* tidak diautentikasi sehingga dapat dimanipulasi, batas peluang diferensial yang dapat dibuktikan baru 2^{-60}, masih ada bias kecil yang bergantung pada posisi bit *round key* (setara dengan kehilangan entropi sekitar 0,007 bit), dan kinerja implementasi Python hanya sekitar 60–70 KB/s.

6.2 Saran Pengembangan (Future Works)

1. **Autentikasi.** *Encrypt-then-MAC* dengan HMAC-SHA256 atas $N \| C$ perlu ditambahkan, atau Pregnant_Sister dapat dikembangkan menjadi mode AEAD. Pesan galat *padding*

juga perlu disatukan agar tidak menjadi *padding oracle*.

2. **KDF untuk kata sandi.** SHA-256 sudah menghilangkan kunci ekuivalen, tetapi SHA-256 adalah fungsi *hash* yang cepat. Untuk kunci yang berasal dari kata sandi, sebaiknya disediakan KDF lambat dengan *salt*, seperti PBKDF2 atau Argon2, agar serangan tebakan kata sandi menjadi mahal.

3. **Ekstraksi *round key* yang lebih seragam.** Bias kecil per posisi bit dapat dihilangkan, misalnya dengan melewatkan grid hasil *whitening* ke Kotak-S atau fungsi *hash* sebelum dilipat, atau dengan mengacak urutan sel pada pengisian spiral.

4. **Analisis formal.** Batas jumlah minimum Kotak-S aktif perlu dihitung dengan MILP untuk kriptanalisis diferensial dan linear. Kajian terhadap serangan *related-key* juga perlu dilakukan, karena *nonce* publik ikut menentukan *round key*. Keluaran *cipher* sebaiknya juga diuji dengan rangkaian uji statistik NIST SP 800-22 [17].

5. **Kinerja dan keamanan implementasi.** Laju sekitar 60–70 KB/s dapat ditingkatkan dengan implementasi bit-sliced dalam C/Rust atau dengan memproses blok secara paralel pada mode ECB dan CTR. Implementasi *constant-time* juga diperlukan, karena *lookup* Kotak-S dan rotasi dengan bilangan bulat Python berpotensi membocorkan informasi melalui waktu eksekusi.

6. **Perbaikan antarmuka.** Mode operasi dapat disimpan di *header ciphertext* agar penerima tidak perlu mengetahuinya terlebih dahulu. Galat validasi kunci sebaiknya dilempar sebagai *exception*, bukan hanya dicetak. *Counter* CTR juga perlu dibungkus modulo 2^{256} untuk menghindari *overflow* pada *nonce* yang sangat besar, walaupun peluang kejadiannya dapat diabaikan.

Daftar Pustaka

[1] M. Bellare dan C. Namprempre, "Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm," dalam *Advances in Cryptology – ASIACRYPT 2000*, LNCS 1976. Springer, 2000, hlm. 531–545.

[2] E. Biham dan A. Shamir, "Differential Cryptanalysis of DES-like Cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, hlm. 3–72, 1991.

[3] J. Daemen dan V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*. Berlin: Springer, 2002.

[4] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST Special Publication 800-38A, 2001.

[5] H. Feistel, "Cryptography and Computer Privacy," *Scientific American*, vol. 228, no. 5, hlm. 15–23, 1973.

[6] M. Gardner, "Mathematical Games: The Fantastic Combinations of John Conway's New Solitaire Game 'Life'," *Scientific American*, vol. 223, no. 4, hlm. 120–123, 1970.

[7] H. M. Heys, "A Tutorial on Linear and Differential Cryptanalysis," *Cryptologia*, vol. 26, no. 3, hlm. 189–221, 2002.

[8] R. Housley, "Cryptographic Message Syntax (CMS)," RFC 5652, IETF, 2009.

[9] H. Krawczyk, M. Bellare, dan R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, 1997.

[10] M. Matsui, "Linear Cryptanalysis Method for DES Cipher," dalam *Advances in Cryptology – EUROCRYPT '93*, LNCS 765. Springer, 1994, hlm. 386–397.

[11] G. A. Miller, "Note on the Bias of Information Estimates," dalam H. Quastler (ed.), *Information Theory in Psychology: Problems and Methods*. Glencoe, IL: Free Press, 1955, hlm. 95–100.

[12] R. Munir, "Block Cipher (Bagian 1)," bahan kuliah IF4020 Kriptografi, Institut Teknologi Bandung, 2026. [Daring]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf

[13] S. Nandi, B. K. Kar, dan P. Pal Chaudhuri, "Theory and Applications of Cellular Automata in Cryptography," *IEEE Transactions on Computers*, vol. 43, no. 12, hlm. 1346–1357, 1994.

[14] National Institute of Standards and Technology, "Data Encryption Standard (DES)," FIPS PUB 46-3, 1999.

[15] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001.

[16] R. L. Rivest, M. J. B. Robshaw, R. Sidney, dan Y. L. Yin, "The RC6 Block Cipher," proposal AES, 1998.

[17] A. Rukhin dkk., "A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," NIST Special Publication 800-22 Rev. 1a, 2010.

[18] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, hlm. 379–423, 1948.

[19] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, hlm. 656–715, 1949.

[20] S. Vaudenay, "Security Flaws Induced by CBC Padding – Applications to SSL, IPSEC, WTLS...," dalam *Advances in Cryptology – EUROCRYPT 2002*, LNCS 2332. Springer, 2002, hlm. 534–545.

[21] C. S. Wallace, "A Suggestion for a Fast Multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, hlm. 14–17, 1964.

[22] A. F. Webster dan S. E. Tavares, "On the Design of S-Boxes," dalam *Advances in Cryptology – CRYPTO '85*, LNCS 218. Springer, 1986, hlm. 523–534.

[23] S. Wolfram, "Cryptography with Cellular Automata," dalam *Advances in Cryptology – CRYPTO '85*, LNCS 218. Springer, 1986, hlm. 429–432.

Lampiran A

Tautan Repositori dan Dokumentasi

- Repositori kode sumber: https://github.com/Cola1000/Pregnant_Sister
- Dokumentasi API: https://cola1000.github.io/Pregnant_Sister/

Lampiran B

Keluaran Pengujian Keamanan

Keluaran lengkap `python -m test.test_security` (benih acak 69420) untuk versi akhir Pregnant_Sister. Semua pemeriksaan berlabel `[OK]`. Baris *S-box is affine* bernilai `False` karena Kotak-S versi akhir bukan lagi Kotak-S afin, dan baris *Keys* $X||X$ menunjukkan bahwa kunci ekuivalen akibat pelipatan sudah tidak ada (Bab 5).

```
[OK] Round-trip in all modes and lengths: 0 failures
[OK] Plaintext avalanche (block level): 49.95% (min 98, max 162 of 256 bits)
[OK] Key avalanche (block level): 50.46% (min 115 bits)
       full diffusion check, 2 rounds: 23.6%
       full diffusion check, 3 rounds: 45.0%
       full diffusion check, 4 rounds: 50.1%
[OK] Raw ciphertext of 64 KB zeros: entropy 7.9973/8, chi2 239.6 (df=255, pass < 293)
       note: base64 output can never exceed 6.0 bits entropy, so measure on raw bytes
       throughput: 68.3 KB/s
[OK] S-box is affine (167x + 91 mod 256): False
[OK] S-box max differential count: 4/256 (AES: 4/256)
[OK] S-box max linear bias: 16/256 (AES: 16/256)
[OK] f_spn: P[0x80..80 -> 0x80..80] = 0/4000 (<= 2^-11.97)
       GoL + 1 SPN + GoL: predicted 2^-3, observed 0 (0/2000)
       GoL + 4 SPN + GoL: predicted 2^-9, observed 0 (0/20000)
[OK] Full 18-round cipher: 11 active SPN rounds -> distinguisher with probability <=
    2^-132 (need < 2^-128)
[OK] Round-key bit density: 0.4985 (ideal 0.5)
[OK] Round-key byte entropy: 7.9934/8 (random bytes, same sample size: ~7.99)
[OK] Keys X||X and 'A'*64 are not interchangeable
```

Kode B.1: Keluaran `test_security`

Lampiran C

Hash SHA-256 Berkas Uji

Hash berikut sama untuk berkas asli dan untuk hasil dekripsi pada kelima mode operasi.

Tabel C.1: *Hash* SHA-256 berkas uji

Berkas	**SHA-256**
glasses.txt	76d3bcab876abc4122b152305d979e769c2aef78c9d0232b07a63de79544e342
image.jpg	5a92b5ae1f92711d77169315198b91b25ea103ee9db38d7e16e4bc8c42b26fee
video.mp4	f8564e7de424642ca2743b781833b3b6ea76da1e7546ca4654186e857cb95436

Lampiran D

Kode Sumber Inti

```
from src.automaton import evolve, bytes_to_grid, grid_to_bytes
from src.spn import f_spn, xor_bytes  # f_spn(half, round_key):

ROUNDS = list(range(1,19)) # list of indices

def f_gol(half, round_key, generations=4) -> bytes:
    # Biar bisa ngetes di komen dulu

    rxor = xor_bytes(half, round_key)

    grid = bytes_to_grid(rxor, rows=8, cols=16)
    evolved = evolve(grid, generations)
    flattened = grid_to_bytes(evolved)

    # additional xor to mitigate GoL collisions
    out = xor_bytes(flattened, half)

    return out
    # return half


def round_function(half, round_key, round_index, total_rounds=18) ->
   bytes:
    if round_index == 1 or round_index == total_rounds:
        return f_gol(half, round_key)
    else:
        return f_spn(half, round_key)



# feistel for encrypt and decrypt
# ref: https://www.youtube.com/live/8l9xAvuGJFo
def process_block(block, round_keys, is_decrypt = False) -> bytes:
    if is_decrypt:
        rounds = ROUNDS[::-1]
    else: rounds = ROUNDS


    # bytes
    l = block[:16]
    r = block[16:]

```

```
    for i in rounds:
        store_r = r
        r = xor_bytes(l, round_function(r, round_keys[i-1], i))
        l = store_r

    return r + l
```

Kode D.1: src/feistel.py

```
import operator
import math

# utils
def xor_bytes(a, b) -> bytes:
    length = min(len(a), len(b))
    if length == 0:
        return b""
    return bytes(map(operator.xor, a[:length], b[:length]))


def bytes_to_matrix(data, size=16) -> list[list]:
    n = math.isqrt(size)
    return [list(data[i:i+n]) for i in range(0, size, n)]

def matrix_to_bytes(m) -> bytes:
    return bytes(b for row in m for b in row) # sum concatenates matrix elements to the empty list



# ref: https://github.com/grwna/cryptosystem-complexity-analysis/blob/main/src/aes_rounds.py

# lookup tables for galois_mult
MUL2 = [0] * 0x100
MUL3 = [0] * 0x100

for i in range(256):
    m2 = (i << 1) ^ 0x1B if (i & 0x80) else (i << 1)
    MUL2[i] = m2 & 0xFF
    MUL3[i] = MUL2[i] ^ i

# precomputed AES sbox (GF(2^8) inverse + affine transform)
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0
```

```
    x21, 0x10, 0xff, 0xf3, 0xd2,
42      0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0
    x3d, 0x64, 0x5d, 0x19, 0x73,
43      0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0
    x14, 0xde, 0x5e, 0x0b, 0xdb,
44      0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0
    x62, 0x91, 0x95, 0xe4, 0x79,
45      0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0
    xea, 0x65, 0x7a, 0xae, 0x08,
46      0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0
    x1f, 0x4b, 0xbd, 0x8b, 0x8a,
47      0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0
    xb9, 0x86, 0xc1, 0x1d, 0x9e,
48      0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0
    xe9, 0xce, 0x55, 0x28, 0xdf,
49      0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0
    x0f, 0xb0, 0x54, 0xbb, 0x16,
50  ]
51
52
53  def sub_bytes(state) -> list[list]:
54      for i in range(len(state)):
55          for j in range(len(state[i])):
56              state[i][j] = SBOX[state[i][j]]
57      return state
58
59
60  def shift_rows(state) -> list[list]:
61      for i in range(1, len(state)):
62          state[i] = state[i][i:] + state[i][:i]
63      return state
64
65
66  def mix_columns(state) -> list[list]:
67      n = len(state) # matrix dim
68      for c in range(n):
69          col = [state[r][c] for r in range(4)]
70          new = [
71              MUL2[col[0]] ^ MUL3[col[1]] ^ col[2] ^ col[3],
72              col[0] ^ MUL2[col[1]] ^ MUL3[col[2]] ^ col[3],
73              col[0] ^ col[1] ^ MUL2[col[2]] ^ MUL3[col[3]],
74              MUL3[col[0]] ^ col[1] ^ col[2] ^ MUL2[col[3]],
75          ]
76          for r in range(n):
77              state[r][c] = new[r]
78      return state
79
80
81  def mod_add(state, round_key) -> bytes:
82      return bytes((a + b) % 256 for a, b in zip(state, round_key))
83
84
85  def rotate(state, amount=0) -> bytes:
86      # works like rc6 rotation, rotation amount depends on data
87      val = int.from_bytes(bytes(state), byteorder='big')
88
89      # 7 bits needed (2^7 = 128)
90      amount = ((state[0] * 5 + state[-1]) ^ state[8]) % 128
```

```
    rot = ((val << amount) | (val >> (128 - amount))) & ((1 << 128) - 1)
    return rot.to_bytes(16, byteorder='big')


def f_spn(half, round_key):
    s = xor_bytes(half, round_key)

    m_s = bytes_to_matrix(s)
    m_s = sub_bytes(m_s)
    m_s = shift_rows(m_s)
    m_s = mix_columns(m_s)

    s = matrix_to_bytes(m_s)
    s = mod_add(s, round_key)
    s = rotate(s)

    return s
```

Kode D.2: src/spn.py

```
import hashlib
import secrets

from src.automaton import evolve, grid_to_bytes
from src.types import Grid


def validate_key(key: str | bytes | bytearray, min_length: int = 32) ->
    bytes:
    if isinstance(key, str):
        key = key.encode("utf-8")
    elif not isinstance(key, (bytes, bytearray)):
        raise TypeError(f"Key must be str or bytes, got {type(key).
    __name__}")

    if len(key) < min_length:
        raise ValueError(f"Key must be at least {min_length} bytes, got
    {len(key)}")

    return bytes(key)


def generate_nonce(size: int = 32) -> bytes:
    return secrets.token_bytes(size)


def shift_nonce(nonce: bytes, amount: int) -> bytes:
    if not nonce:
        return b""
    shift = amount % len(nonce)
    return nonce[shift:] + nonce[:shift]


def rotate_byte(b: int, rot: int) -> int:
    rot = rot % 8
    return ((b << rot) & 0xFF) | (b >> (8 - rot))


```

```
def mix_key(key: str | bytes | bytearray, nonce: bytes) -> bytes:
    key_bytes = validate_key(key)

    # hash key into 32 bytes
    key_material = hashlib.sha256(key_bytes).digest()

    # mix with nonce bytes via rotation and xor
    mixed = bytearray(32)
    nonce_len = len(nonce)
    for i in range(32):
        s = nonce[i % nonce_len] if nonce_len > 0 else 0
        mixed[i] = rotate_byte(key_material[i], s % 8) ^ s

    return bytes(mixed)


def fill_grid(material: bytes, rows: int = 16, cols: int = 16) -> list[
    list[int]]:
    grid = [[0] * cols for _ in range(rows)]
    mat_len = len(material)
    if mat_len == 0:
        return grid

    top, bottom = 0, rows - 1
    left, right = 0, cols - 1
    idx = 0

    while top <= bottom and left <= right:
        for c in range(left, right + 1):
            lap = idx // mat_len
            grid[top][c] = (material[idx % mat_len] + 31 * lap) % 256
            idx += 1
        top += 1

        for r in range(top, bottom + 1):
            lap = idx // mat_len
            grid[r][right] = (material[idx % mat_len] + 31 * lap) % 256
            idx += 1
        right -= 1

        if top <= bottom:
            for c in range(right, left - 1, -1):
                lap = idx // mat_len
                grid[bottom][c] = (material[idx % mat_len] + 31 * lap) %
    256
                idx += 1
            bottom -= 1

        if left <= right:
            for r in range(bottom, top - 1, -1):
                lap = idx // mat_len
                grid[r][left] = (material[idx % mat_len] + 31 * lap) %
   256
                idx += 1
            left += 1

    return grid

```

```python

def parity_grid(grid: list[list[int]]) -> list[list[int]]:
    return [[val.bit_count() % 2 for val in row] for row in grid]


def key_pattern(key: str | bytes | bytearray, nonce: bytes) -> list[list[int]]:
    material = mix_key(key, nonce)
    grid = fill_grid(material, rows=16, cols=16)
    return parity_grid(grid)


def matrix_to_grid(matrix: list[list[int]]) -> Grid:
    rows = len(matrix)
    cols = len(matrix[0]) if rows > 0 else 0
    grid_rows = []
    for row in matrix:
        val = 0
        for bit in row:
            val = (val << 1) | (bit & 1)
        grid_rows.append(val)
    return Grid(grid_rows, rows, cols)


def grid_to_matrix(grid: Grid) -> list[list[int]]:
    return [
        [(r >> c) & 1 for c in range(grid.num_cols - 1, -1, -1)]
        for r in grid.rows
    ]


def xor_grids(a: Grid | list[list[int]], b: Grid | list[list[int]]) -> Grid | list[list[int]]:
    if isinstance(a, Grid) and isinstance(b, Grid):
        return Grid(
            rows=[r1 ^ r2 for r1, r2 in zip(a.rows, b.rows)],
            num_rows=a.num_rows,
            num_cols=a.num_cols,
        )
    if isinstance(a, Grid):
        a = grid_to_matrix(a)
    if isinstance(b, Grid):
        b = grid_to_matrix(b)
    return [
        [c1 ^ c2 for c1, c2 in zip(r1, r2)]
        for r1, r2 in zip(a, b)
    ]


def fold_bytes(data: bytes) -> bytes:
    half = len(data) // 2
    return bytes(a ^ b for a, b in zip(data[:half], data[half:half * 2]))


def derive_round_keys(
    key: str | bytes | bytearray,
    nonce: bytes,
```

```
    rounds: int = 18,
    generations_per_round: int = 4,
) -> list[bytes]:
    key_bytes = validate_key(key)
    initial_pattern = key_pattern(key_bytes, nonce)
    grid = matrix_to_grid(initial_pattern)

    round_keys = []
    for r in range(rounds):
        # shifted nonce reinjection with round perturbation
        shifted_nonce = shift_nonce(nonce, r + 1)
        reinj_nonce = bytes((b ^ (((r + 1) * 17) & 0xFF)) for b in
   shifted_nonce)
        reinj_grid = matrix_to_grid(key_pattern(key_bytes, reinj_nonce))
        grid = xor_grids(grid, reinj_grid)

        # Conway's GoL evolution on toroidal grid
        grid = evolve(grid, generations_per_round)

        # 32 bytes (256-bit grid) folded to 16 bytes (128-bit round key)
        whitened_bytes = grid_to_bytes(xor_grids(grid, reinj_grid))
        round_keys.append(fold_bytes(whitened_bytes))

    return round_keys
```

Kode D.3: src/keygen.py

```
from src.types import Grid


# Toroidal Shifting for Horizontal
def rotl(row: int, n: int) -> int:
    mask = (1 << n) - 1
    # wrapping to pos 0 after overflow
    msb = (row >> (n - 1)) & 1
    return ((row << 1) & mask) | msb

def rotr(row: int, n: int) -> int:
    mask = (1 << n) - 1
    # wrapping to pos col - 1 after overflow
    lsb = row & 1
    return ((row >> 1) & mask) | (lsb << (n - 1))

# Adders
def half_adder(a: int, b: int) -> tuple[int, int]:
    s = a ^ b
    c = a & b
    return s, c

def full_adder(a: int, b:int, c: int) -> tuple[int, int]:
    s = a ^ b ^ c
    carry = (a & b) | (c & (a ^ b))
    return s, carry

# Wallace Tree
def count_neighbors(n1, n2, n3, n4, n5, n6, n7, n8) -> tuple[int, int,
   int]:
    s1, c1 = full_adder(n1, n2, n3)
    s2, c2 = full_adder(n4, n5, n6)
```

```
    s3, c3 = half_adder(n7, n8)

    c0, c_extra = full_adder(s1, s2, s3)
    s_c, c_c = full_adder(c1, c2, c3)

    c1, c_high = half_adder(s_c, c_extra)

    c2_or_more = c_c | c_high

    return c0, c1, c2_or_more

# GoL B3/S23 Rule
def next_state(current: int, c0: int, c1: int, c2_or_more: int) -> int:
    # Rules
    # Cell only lives if total neighbor count is 2 or 3 (less than 2 under population, more than 3 overpopulation)
    # 2 in binary is 010 (c1=1, c0=0, c2=0)
    # 3 in binary is 011 (c1=1, c0=1, c2=0)

    count_2_or_3 = c1 & (~c2_or_more)
    count_3 = count_2_or_3 & c0

    # New cell if count is 3 or survival if alive and count is 2 or 3
    return count_3 | (current & count_2_or_3)

def step(grid: Grid) -> Grid:
    m = grid.num_rows
    new_rows = []
    for r in range(m):
        top = grid.rows[(r - 1) % m]
        mid = grid.rows[r]
        bot = grid.rows[(r + 1) % m]

        n_nw = rotl(top, grid.num_cols); n_n = top; n_ne = rotr(top, grid.num_cols)
        n_w  = rotl(mid, grid.num_cols);            n_e  = rotr(mid, grid.num_cols)
        n_sw = rotl(bot, grid.num_cols); n_s = bot; n_se = rotr(bot, grid.num_cols)

        c0, c1, c2_or_more = count_neighbors(
            n_nw, n_n, n_ne,
            n_w,       n_e,
            n_sw, n_s, n_se
        )
        new_rows.append(next_state(mid, c0, c1, c2_or_more))

    return Grid(new_rows, grid.num_rows, grid.num_cols)

def evolve(grid: Grid, generations: int) -> Grid:
    current_grid = grid
    for _ in range(generations):
        current_grid = step(current_grid)
    return current_grid

def bytes_to_grid(data: bytes, rows, cols: int) -> Grid:
    total_bits = int.from_bytes(data, byteorder="big")

```

```
    grid_rows = []
    for r in range(rows):
        shift = (rows - 1 - r) * cols
        row_val = (total_bits >> shift) & ((1 << cols) -1)
        grid_rows.append(row_val)

    return Grid(grid_rows, rows, cols)

def grid_to_bytes(grid: Grid) -> bytes:
    total_bits = 0
    for r in grid.rows:
        total_bits = (total_bits << grid.num_cols) | r

    total_bytes = (len(grid.rows) * grid.num_cols + 7) // 8
    return total_bits.to_bytes(total_bytes, byteorder="big")
```

Kode D.4: src/automaton.py

```
from src.feistel import process_block
from src.spn import xor_bytes

def split_blocks(data, block_size=32) -> list[bytes]:
    return [data[i:i + block_size] for i in range(0, len(data),
    block_size)]

# each mode functions returns bytes
# ref:
# - https://www.geeksforgeeks.org/ethical-hacking/block-cipher-modes-of-
    operation/
# - https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi
    /2026-2027/08-Block-Cipher-Bagian1-(2026).pdf
def ecb_encrypt(blocks, round_keys, is_decrypt=False):
    outs = []
    for pt in blocks:
        outs.append(process_block(pt, round_keys, is_decrypt))
    return b"".join(outs)


def ecb_decrypt(blocks, round_keys):
    return ecb_encrypt(blocks, round_keys, is_decrypt=True)


def cbc_encrypt(blocks, round_keys, iv):
    outs = []
    fb = iv # feedback value
    for pt in blocks:
        xored = xor_bytes(fb, pt)
        fb = process_block(xored, round_keys)
        outs.append(fb)

    return b"".join(outs)



def cbc_decrypt(blocks, round_keys, iv):
    outs = []
    fb = iv
    for ct in blocks:
        decrypted = process_block(ct, round_keys, is_decrypt=True)
```

```
39          outs.append(xor_bytes(fb, decrypted))
40          fb = ct
41      return b"".join(outs)
42
43
44  # full block cfb
45  def cfb_encrypt(blocks, round_keys, iv):
46      outs = []
47      fb = iv
48      for pt in blocks:
49          enc = process_block(fb, round_keys)
50          fb = xor_bytes(pt, enc)
51          outs.append(fb)
52      return b"".join(outs)
53
54
55  def cfb_decrypt(blocks, round_keys, iv):
56      outs = []
57      fb = iv
58      for ct in blocks:
59          enc = process_block(fb, round_keys)
60          outs.append(xor_bytes(ct, enc))
61          fb = ct
62      return b"".join(outs)
63
64
65
66  def ofb_encrypt(blocks, round_keys, iv):
67      outs = []
68      fb = iv
69      for pt in blocks:
70          fb = process_block(fb, round_keys)
71          outs.append(xor_bytes(fb, pt))
72      return b"".join(outs)
73
74
75  def ofb_decrypt(blocks, round_keys, iv):
76      return ofb_encrypt(blocks, round_keys, iv)
77
78
79  def ctr_encrypt(blocks, round_keys, nonce):
80      outs = []
81      ctr = int.from_bytes(nonce, byteorder="big")
82      for pt in blocks:
83          ctr_bytes = ctr.to_bytes(32, byteorder="big")
84          enc = process_block(ctr_bytes, round_keys)
85          outs.append(xor_bytes(pt, enc))
86          ctr += 1
87      return b"".join(outs)
88
89
90  def ctr_decrypt(blocks, round_keys, nonce):
91      return ctr_encrypt(blocks, round_keys, nonce)
92
93
94  # split full data into 32-byte chunks (blocks)
95  # process each blocks according to the mode
96  def encrypt_blocks(data, round_keys, mode, nonce):
```

```
    blocks = split_blocks(data)

    match mode.upper():
        case "ECB":
            return ecb_encrypt(blocks, round_keys)
        case "CBC":
            return cbc_encrypt(blocks, round_keys, nonce)
        case "CFB":
            return cfb_encrypt(blocks, round_keys, nonce)
        case "OFB":
            return ofb_encrypt(blocks, round_keys, nonce)
        case "CTR":
            return ctr_encrypt(blocks, round_keys, nonce)


def decrypt_blocks(data, round_keys, mode, nonce):
    blocks = split_blocks(data)

    match mode.upper():
        case "ECB":
            return ecb_decrypt(blocks, round_keys)
        case "CBC":
            return cbc_decrypt(blocks, round_keys, nonce)
        case "CFB":
            return cfb_decrypt(blocks, round_keys, nonce)
        case "OFB":
            return ofb_decrypt(blocks, round_keys, nonce)
        case "CTR":
            return ctr_decrypt(blocks, round_keys, nonce)
```

Kode D.5: src/modes.py

```
# PKCS#7 with slight mod
# ref: https://pentesterlab.com/glossary/pkcs7-padding
def pad(data, block_size=32):
    p = block_size - (len(data) % block_size )
    return data + (p.to_bytes(1, "big") * p)



def unpad(data, block_size=32):
    if not data or len(data) % block_size != 0:
        print("Unpad fail: data invalid") # TODO: error msg
        return b""

    p = data[-1]

    if p < 1 or p > block_size:
        raise ValueError("Invalid padding length")

    if data[-p:] != bytes([p]) * p:
        raise ValueError("Invalid padding sequence")

    return data[:-p]
```

Kode D.6: src/padding.py

```
import base64

```

```python
from src.keygen import validate_key, generate_nonce, derive_round_keys
from src.padding import pad
from src.modes import encrypt_blocks

def convert_bytes(data) -> bytes:
    if not isinstance(data, (bytes, bytearray)):
        if isinstance(data, str):
            return data.encode("utf-8")
        else:
            return bytes(data)
    return data


def encrypt(data, key, mode="CBC", nonce_size=32) -> bytes:
    try:
        validate_key(key)
    except:
        print("Error: invalid key") # TODO: error msg

    data = convert_bytes(data)
    data = pad(data)

    nonce = generate_nonce()
    round_keys = derive_round_keys(key, nonce)

    ct = encrypt_blocks(data, round_keys, mode, nonce)
    ct = nonce + ct

    return base64.b64encode(ct)
```

Kode D.7: src/encrypt.py

```python
import base64

from src.keygen import validate_key, derive_round_keys
from src.padding import unpad
from src.modes import decrypt_blocks
from src.encrypt import convert_bytes


def decrypt(data, key, mode="CBC", nonce_size=32) -> bytes:
    try:
        validate_key(key)
    except:
        print("Error: invalid key") # TODO: error msg

    data = convert_bytes(data)

    decoded = base64.b64decode(data) # ct
    nonce = decoded[:nonce_size]
    round_keys = derive_round_keys(key, nonce)
    pt = decrypt_blocks(decoded[nonce_size:], round_keys, mode, nonce)
    return unpad(pt)
```

Kode D.8: src/decrypt.py